Universidade Estadual de Santa Cruz – UESC

Relatórios de Implementações de Métodos da Disciplina Análise Numérica

Relatório de implementações realizadas por Ronaldo Ribeiro Porto Filho

Disciplina Análise Numérica.

Curso Ciência da Computação

Semestre 2026.1

Professor Gesil Sampaio Amarante II

Ilhéus – BA
2026

ÍNDICE

ÍNDICE **2**
Linguagem(ns) Escolhida(s) e justificativas **5**
Método da Bissecção **6**
Estratégia de Implementação: 6
Estrutura dos Arquivos de Entrada/Saída 6
Dificuldades enfrentadas 6
Método da Falsa Posição **8**
Estratégia de Implementação: 8
Estrutura dos Arquivos de Entrada/Saída 8
Dificuldades enfrentadas 8
Método de Newton-Raphson **9**
Estratégia de Implementação: 9
Estrutura dos Arquivos de Entrada/Saída 9
Dificuldades enfrentadas 10
Método da Secante **11**
Estratégia de Implementação: 11
Estrutura dos Arquivos de Entrada/Saída 11
Dificuldades enfrentadas 11
Problema Teste 1 - Página 97 Questão 3.3 13
Entrada 1 – para Bissecção, Falsa Posição e Secante 13
Entrada 1 – para Newton-Raphson 13
Entrada 2 – para Bissecção, Falsa Posição e Secante 13
Entrada 2 – para Newton-Raphson 13
Saída – Bissecção (Entrada 1) 14
Saída – Falsa Posição (Entrada 1) 15
Saída – Newton-Raphson (Entrada 1) 15
Saída – Secante (Entrada 1) 16
Saída – Bissecção (Entrada 2) 16
Saída – Falsa Posição (Entrada 2) 17
Saída – Newton-Raphson (Entrada 2) 17
Saída – Secante (Entrada 2) 18
Problema Teste 2 - Página 100 Questão 3.6 19
Entrada 1 – para Bissecção, Falsa Posição e Secante 19
Entrada 1 – para Newton-Raphson 19
Saída – Bissecção (Entrada 1) 19
Saída – Falsa Posição (Entrada 1) 20
Saída – Newton-Raphson (Entrada 1) 20
Saída – Secante (Entrada 1) 20
Problema Teste 3 - Página 100 Questão 3.8 21

Entrada 1 – para Bissecção, Falsa Posição e Secante..... 21
Entrada 1 – para Newton-Raphson..... 21
Entrada 2 – para Bissecção, Falsa Posição e Secante..... 21
Entrada 2 – para Newton-Raphson..... 21
Saída – Bissecção (Entrada 1) – Plano A.....22
Saída – Falsa Posição (Entrada 1) – Plano A..... 23
Saída – Newton-Raphson (Entrada 1) – Plano A..... 23
Saída – Secante (Entrada 1) – Plano A.....24
Saída – Bissecção (Entrada 2) – Plano B.....24
Saída – Falsa Posição (Entrada 2) – Plano B..... 25
Saída – Newton-Raphson (Entrada 2) – Plano B..... 25
Saída – Secante (Entrada 2) – Plano B.....26
Método da Eliminação de Gauss.....27
Estratégia de Implementação:..... 27
Estrutura dos Arquivos de Entrada/Saída..... 27
Dificuldades enfrentadas..... 27
Método da Fatoração LU.....29
Estratégia de Implementação:..... 29
Estrutura dos Arquivos de Entrada/Saída..... 29
Dificuldades enfrentadas..... 30
Problema Teste 1 - Página 150 Questão 4.1..... 31
Entrada 1 – para Eliminação de Gauss e Fatoração LU.....31
Saída – Eliminação de Gauss (Entrada 1)..... 32
Saída – Fatoração LU (Entrada 1)..... 33
Problema Teste 2 - Página 151 Questão 4.3..... 34
Entrada 1 – para Eliminação de Gauss e Fatoração LU.....34
Saída – Eliminação de Gauss (Entrada 1)..... 35
Saída – Fatoração LU (Entrada 1)..... 37
Problema Teste 3 - Página 153 Questão 4.6..... 39
Entrada 1 – para Eliminação de Gauss e Fatoração LU.....39
Saída – Eliminação de Gauss (Entrada 1)..... 39
Saída – Fatoração LU (Entrada 1)..... 40
Método de Gauss-Seidel.....41
Estratégia de Implementação:..... 41
Estrutura dos Arquivos de Entrada/Saída..... 41
Dificuldades enfrentadas..... 42
Método de Jacobi.....43
Estratégia de Implementação:..... 43
Estrutura dos Arquivos de Entrada/Saída..... 43
Dificuldades enfrentadas..... 43
Condicionamento de Matrizes.....45
Estratégia de Implementação:..... 45

Estrutura dos Arquivos de Entrada/Saída..... 45
Dificuldades enfrentadas..... 46
Problema Teste 1 - Página 185 Questão 5.1..... 47
Entrada 1 – para Gauss-Seidel e Jacobi..... 47
Entrada 1 – para Condicionamento de Matriz..... 47
Saída – Gauss-Seidel (Entrada 1)..... 48
Saída – Jacobi (Entrada 1)..... 51
Saída – Condicionamento de Matriz (Entrada 1)..... 53
Problema Teste 2 - Página 185 Questão 5.2..... 54
Entrada 1 – para Gauss-Seidel e Jacobi..... 54
Entrada 1 – para Condicionamento de Matriz..... 54
Saída – Gauss-Seidel (Entrada 1)..... 55
Saída – Jacobi (Entrada 1)..... 57
Saída – Condicionamento de Matriz (Entrada 1)..... 58
Problema Teste 3 - Página 188 Questão 5.5..... 60
Entrada 1 – para Gauss-Seidel e Jacobi..... 60
Entrada 1 – para Condicionamento de Matriz..... 61
Saída – Gauss-Seidel (Entrada 1)..... 62
Saída – Jacobi (Entrada 1)..... 66
Saída – Condicionamento de Matriz (Entrada 1)..... 69
Considerações Finais..... 70

Linguagem(ns) Escolhida(s) e justificativas

A linguagem de programação escolhida para o desenvolvimento de todos os métodos numéricos deste projeto foi o Python.

A escolha do Python justifica-se, primordialmente, pela sua sintaxe clara e alta legibilidade, características que aceleram o processo de prototipagem matemática e a tradução direta de pseudocódigos e fórmulas analíticas para blocos de código funcional. A linguagem possui uma biblioteca padrão rica; a utilização do módulo interno "math" permitiu o acesso direto a funções trigonométricas e constantes sem a necessidade de dependências externas pesadas.

Uma ferramenta de extrema importância empregada na solução foi a função nativa "eval()", combinada com expressões "lambda". Isso solucionou o desafio de ler uma função matemática arbitrária como texto a partir de um arquivo e transformá-la em um objeto executável em tempo de execução.

Em termos de limitações, o Python é uma linguagem interpretada e dinamicamente tipada, o que o torna inerentemente mais lento em tempo de execução quando comparado a linguagens compiladas nativas. No entanto, para o escopo destes algoritmos de busca de raízes, o custo computacional é muito baixo, tornando essa limitação de desempenho insignificante frente ao ganho em velocidade de desenvolvimento.

Método da Bissecção

Estratégia de Implementação:

A implementação foi desenvolvida com foco na confiabilidade e prevenção de loops infinitos. Para superar os desafios da identificação da raiz, foram implementados os seguintes tratamentos e decisões algorítmicas:

(a) Verificação de validade do intervalo: Logo no início, o algoritmo testa a condição de mudança de sinal: f(xl) * f(xu) >= 0. Se a função não mudar de sinal nas extremidades, o programa acusa erro e encerra, impedindo cálculos inválidos.

(b) Critérios de parada duplos: O loop principal é controlado pelo erro relativo percentual aproximado (ea). O laço continua enquanto "ea" for maior que a tolerância (es). Como fator de segurança contra casos de não-convergência, introduziu-se um limite máximo de 100 iterações.

(c) Cálculo do erro: Para evitar divisões por zero e erros na primeira iteração (onde não há um valor anterior para comparar), o erro inicial foi fixado artificialmente em 100%, e o cálculo formal só ocorre quando a nova raiz aproxima-se de valores diferentes de zero.

Estrutura dos Arquivos de Entrada/Saída

Optou-se por separar a lógica matemática dos dados de uso genérico.

(1) Arquivo de Entrada (entrada.txt): Estruturado de forma estrita em linhas para simplificar a leitura. A primeira linha contém a função formatada para a biblioteca math, seguida pelo limite inferior (xl), o limite superior (xu) e a tolerância de erro percentual (es).

(2) Arquivo de Saída (saida.txt): Foi formatado como uma tabela de log (Iteração, xl, xu, xr e ea%). Essa escolha é fundamental na Análise Numérica, pois permite a auditoria visual do comportamento do algoritmo passo a passo, comprovando a convergência gradual até o resultado final.

Dificuldades enfrentadas

As dificuldades gerais na aplicação deste método costumam estar na definição do intervalo inicial. O método exige que o analista saiba previamente

onde a raiz está confinada, o que pode ser trabalhoso sem uma análise gráfica prévia da função. Além disso, a convergência do Método da Bissecção é notavelmente lenta (linear), o que significa que o algoritmo requer um número elevado de iterações, cortando o intervalo ao meio repetidas vezes de forma "cega", sem levar em consideração a inclinação da curva da função.

Método da Falsa Posição

Estratégia de Implementação:

A lógica de controle (verificação de sinais e critérios de parada de tolerância/iterações) foi reaproveitada da estrutura do Método da Bissecção, garantindo consistência no projeto. A principal diferença foi a implementação da fórmula central do método:

xr = xu - [f(xu) * (xl - xu)] / [f(xl) - f(xu)]

Para contornar potenciais limitações matemáticas que causariam falhas no programa, introduziu-se um tratamento específico: a verificação do denominador. Se a subtração f(xl) - f(xu) for igual a zero (o que indicaria uma reta secante horizontal), o código reporta "Divisão por zero encontrada" e interrompe a execução, evitando a quebra do script.

Estrutura dos Arquivos de Entrada/Saída

Manteve-se o exato mesmo padrão de padronização arquitetural (os arquivos entrada.txt e saida.txt). A justificativa é a interoperabilidade: poder usar o mesmo arquivo de entrada para testar diferentes métodos, permitindo a comparação direta de desempenho (número de iterações para alcançar a mesma tolerância) entre Bissecção e Falsa Posição de maneira imediata.

Dificuldades enfrentadas

Ao testar e analisar o Método da Falsa Posição, a principal dificuldade teórica encontrada é o problema da "estagnação" de um dos limites. Se a função for altamente côncava ou convexa nas proximidades da raiz, um dos limites (xl ou xu) pode permanecer fixo durante quase todas as iterações, enquanto o outro limite avança muito lentamente. Isso faz com que o erro aproximado calculado se torne muito pequeno antes mesmo do valor da raiz estar realmente preciso, gerando uma falsa sensação de exatidão e tornando o método, em certos casos, mais lento que a Bissecção tradicional.

Método de Newton-Raphson

Estratégia de Implementação:

Diferente dos métodos de confinamento, o Newton-Raphson é um método aberto que requer apenas um palpite inicial (x0) e o uso obrigatório da derivada da função principal. A fórmula iterativa central implementada no código foi:

x_novo = x_atual - [f(x_atual) / df(x_atual)]

Para garantir a robustez do algoritmo, foram implementadas duas defesas vitais:

(a) Prevenção de inclinação nula: Antes de aplicar a fórmula, o código verifica se df(x_atual) é igual a zero. Se a derivada zerar, significa que a reta tangente é perfeitamente horizontal (um pico ou vale na função), o que causaria um erro de divisão por zero. O programa acusa "Erro Crítico" e encerra o laço iterativo de forma segura.

(b) Verificação final de divergência: Adicionou-se uma checagem pós-loop avaliando se o valor absoluto de f(x_atual) é maior que 0.1. Isso previne o falso positivo gerado quando o erro (ea) fica pequeno simplesmente porque a inclinação está jogando o valor muito longe, alertando o usuário caso a função não tenha se aproximado de fato de zero.

Estrutura dos Arquivos de Entrada/Saída

A estrutura do arquivo "entrada.txt" foi adaptada para conter quatro linhas lidas de forma estrita: a expressão da função f(x), a expressão da sua derivada analítica f'(x), o valor do chute inicial (x0) e a tolerância exigida. No arquivo "saida.txt", o relatório tabular registra (Iteracao | x_atual | f(x) | df(x) | ea %), permitindo visualizar diretamente o comportamento da derivada e a rápida queda no valor da função e do erro, evidenciando a convergência quadrática típica deste método.

Dificuldades enfrentadas

A maior limitação operacional deste método é a exigência do cálculo analítico prévio da derivada, o que deve ser inserido corretamente no arquivo de texto. Do ponto de vista numérico, a principal dificuldade é o risco de divergência dependendo do chute inicial. Como destacado no comportamento do algoritmo, se a inclinação for próxima de zero, a reta tangente pode jogar o próximo palpite iterativo para muito longe da raiz verdadeira, ou até mesmo ficar oscilando os valores de um lado para o outro infinitamente.

Método da Secante

Estratégia de Implementação:

O Método da Secante foi implementado como uma alternativa aberta que elimina a necessidade de calcular a derivada analítica, aproximando-a através de uma reta secante entre os dois pontos iterativos mais recentes. A fórmula implementada foi:

x_novo = x_atual - [f(x_atual) * (x_ant - x_atual)] / [f(x_ant) - f(x_atual)]

O grande desafio de código neste método é o gerenciamento das variáveis de estado. A cada iteração, os valores precisam ser atualizados em uma sequência estrita: o valor antigo recebe o atual, e o valor atual recebe o novo palpite. Além disso, assim como no Newton-Raphson, foi mantida a verificação para evitar divisão por zero (caso o denominador f_ant - f_atual zere, indicando uma reta secante horizontal) e a importante verificação de segurança no final do processamento para alertar sobre raízes falsas caso abs(f(x_atual)) seja muito distante de zero.

Estrutura dos Arquivos de Entrada/Saída

O "entrada.txt" recebe quatro linhas correspondentes à função matemática, aos dois chutes iniciais (x0 e x1) e à tolerância de erro percentual. O diferencial é que x0 e x1 não precisam apresentar sinais opostos na função, diferentemente da Bissecção. O "saida.txt" gera um relatório contendo (Iteracao | x_ant | x_atual | x_novo | ea %), evidenciando de forma muito clara como os pontos vão se substituindo mutuamente para traçar as novas secantes.

Dificuldades enfrentadas

A maior dificuldade intrínseca ao Método da Secante é que ele não verifica sinais (não confina a raiz), o que o torna mais rápido, mas consideravelmente mais instável. Em comparação com o Método da Falsa Posição (que também usa secantes, mas sempre mantém os sinais opostos), a Secante apresenta um risco inerente de divergir para o infinito dependendo do

formato da função e dos chutes iniciais escolhidos. É um método de tentativa mais "cega", exigindo palpites razoavelmente próximos à raiz verdadeira para garantir a convergência.

Problema Teste 1 - Página 97 Questão 3.3

3.3 - *Um amplificador eletrônico com acoplamento R - C com três estágios em cascata tem uma resposta a um degrau unitário de tensão dada pela expressão:*

$$g(T) = 1 - (1 + T + \frac{T^2}{2})e^{-T} \ ,$$

onde $T = \frac{t}{RC}$ *é uma unidade de tempo normalizada. O tempo de subida de um amplificador é definido como o tempo necessário para sua resposta ir de* 10% *a* 90% *de seu valor final. No caso, como* $g(\infty) = 1$ *é necessário calcular os valores de* T *para os quais*

$$g = 0.1 \quad e \quad g = 0.9$$

ou seja resolver as equações:

$$\begin{aligned} 0.1 &= 1 - (1 + T + \frac{T^2}{2})e^{-T} \ . \\ 0.9 &= 1 - (1 + T + \frac{T^2}{2})e^{-T} \ . \end{aligned}$$

Chamando de $T_{0.1}$ *o valor obtido de* T *na* 1^a *equação e* $T_{0.9}$ *o valor obtido de* T *na* 2^a *equação, calcular o tempo de subida.*

Entrada 1 – para Bissecção, Falsa Posição e Secante

```
0.9 - (1 + x + (x**2)/2) * math.exp(-x)
1
2
0.001
```

Entrada 1 – para Newton-Raphson

```
0.9 - (1 + x + (x**2)/2) * math.exp(-x)
((x**2) / 2) * math.exp(-x)
1.5
0.001
```

Entrada 2 – para Bissecção, Falsa Posição e Secante

```
0.1 - (1 + x + (x**2)/2) * math.exp(-x)
5
6
0.001
```

Entrada 2 – para Newton-Raphson

```
0.1 - (1 + x + (x**2)/2) * math.exp(-x)
((x**2) / 2) * math.exp(-x)
5.5
0.001
```

Saída – Bissecção (Entrada 1)

```
====== RELATORIO ======
Funcao analisada: f(x) = 0.9 - (1 + x + (x**2)/2) * math.exp(-x)
Intervalo inicial: [1.0, 2.0]
Tolerancia exigida: 0.001%
=======================

METODO DA BISSECCAO
(Iteracao) | (xl)       | (xu)       | (xr)       | (ea %)
-------------------------------------------------------------------
( 01 )      | (1.000000) | (2.000000) | (1.500000) | (33.3333%)
( 02 )      | (1.000000) | (1.500000) | (1.250000) | (20.0000%)
( 03 )      | (1.000000) | (1.250000) | (1.125000) | (11.1111%)
( 04 )      | (1.000000) | (1.125000) | (1.062500) | (5.8824%)
( 05 )      | (1.062500) | (1.125000) | (1.093750) | (2.8571%)
( 06 )      | (1.093750) | (1.125000) | (1.109375) | (1.4085%)
( 07 )      | (1.093750) | (1.109375) | (1.101562) | (0.7092%)
( 08 )      | (1.101562) | (1.109375) | (1.105469) | (0.3534%)
( 09 )      | (1.101562) | (1.105469) | (1.103516) | (0.1770%)
( 10 )      | (1.101562) | (1.103516) | (1.102539) | (0.0886%)
( 11 )      | (1.101562) | (1.102539) | (1.102051) | (0.0443%)
( 12 )      | (1.102051) | (1.102539) | (1.102295) | (0.0221%)
( 13 )      | (1.102051) | (1.102295) | (1.102173) | (0.0111%)
( 14 )      | (1.102051) | (1.102173) | (1.102112) | (0.0055%)
( 15 )      | (1.102051) | (1.102112) | (1.102081) | (0.0028%)
( 16 )      | (1.102051) | (1.102081) | (1.102066) | (0.0014%)
( 17 )      | (1.102051) | (1.102066) | (1.102058) | (0.0007%)
-------------------------------------------------------------------
Raiz encontrada: 1.102058 com erro de 0.0007% apos 17 iteracoes.
```

Saída – Falsa Posição (Entrada 1)

```
====== RELATORIO ======
Funcao analisada: f(x) = 0.9 - (1 + x + (x**2)/2) * math.exp(-x)
Intervalo inicial: [1.0, 2.0]
Tolerancia exigida: 0.001%
=======================

METODO DA FALSA POSICAO
(Iteracao) | (xl)       | (xu)       | (xr)       | (ea %)
-----------------------------------------------------------------
( 01 )      | (1.000000) | (2.000000) | (1.081057) | (100.0000%)
( 02 )      | (1.081057) | (2.000000) | (1.098026) | (1.5454%)
( 03 )      | (1.098026) | (2.000000) | (1.101300) | (0.2973%)
( 04 )      | (1.101300) | (2.000000) | (1.101921) | (0.0564%)
( 05 )      | (1.101921) | (2.000000) | (1.102038) | (0.0107%)
( 06 )      | (1.102038) | (2.000000) | (1.102060) | (0.0020%)
( 07 )      | (1.102060) | (2.000000) | (1.102064) | (0.0004%)
-----------------------------------------------------------------
Raiz encontrada: 1.102064 com erro de 0.0004% apos 7 iteracoes.
```

Saída – Newton-Raphson (Entrada 1)

```
====== RELATORIO DE ANALISE NUMERICA ======
Funcao original : f(x)  = 0.9 - (1 + x + (x**2)/2) * math.exp(-x)
Derivada        : f'(x) = ((x**2) / 2) * math.exp(-x)
Chute inicial   : x0 = 1.5
Tolerancia (ea) : 0.001%
==========================================

METODO DE NEWTON-RAPHSON
(Iteracao) | (x_atual)  | (f(x))     | (df(x))    | (ea %)
-----------------------------------------------------------------
( 01 )      | (1.500000) | (0.091153) | (0.251021) | (31.941095%)
( 02 )      | (1.136871) | (0.007119) | (0.207327) | (3.114555%)
( 03 )      | (1.102532) | (0.000094) | (0.201804) | (0.042337%)
( 04 )      | (1.102065) | (0.000000) | (0.201727) | (0.000008%)
( 05 )      | (1.102065) | (0.000000) | (0.201727) | (0.000008%)
-----------------------------------------------------------------
Raiz encontrada: 1.10206533 com erro de 0.000008% apos 4 iteracoes.
```

Saída – Secante (Entrada 1)

```
====== RELATORIO DE ANALISE NUMERICA ======
Funcao analisada : f(x) = 0.9 - (1 + x + (x**2)/2) * math.exp(-x)
Chutes iniciais  : x0 = 1.0, x1 = 2.0
Tolerancia (ea)  : 0.001%
==========================================

METODO DA SECANTE
(Iteracao) | (x_ant)     | (x_atual)   | (x_novo)    | (ea %)
-----------------------------------------------------------------
( 01 )      | (1.000000) | (2.000000) | (1.081057) | (85.004154%)
( 02 )      | (2.000000) | (1.081057) | (1.098026) | (1.545403%)
( 03 )      | (1.081057) | (1.098026) | (1.102101) | (0.369738%)
( 04 )      | (1.098026) | (1.102101) | (1.102065) | (0.003207%)
( 05 )      | (1.102101) | (1.102065) | (1.102065) | (0.000005%)
-----------------------------------------------------------------
Raiz encontrada: 1.10206533 com erro de 0.000005% apos 5 iteracoes.
```

Saída – Bissecção (Entrada 2)

```
====== RELATORIO ======
Funcao analisada: f(x) = 0.1 - (1 + x + (x**2)/2) * math.exp(-x)
Intervalo inicial: [5.0, 6.0]
Tolerancia exigida: 0.001%
=======================

METODO DA BISSECCAO
(Iteracao) | (xl)        | (xu)        | (xr)        | (ea %)
-------------------------------------------------------------------
( 01 )      | (5.000000) | (6.000000) | (5.500000) | (9.0909%)
( 02 )      | (5.000000) | (5.500000) | (5.250000) | (4.7619%)
( 03 )      | (5.250000) | (5.500000) | (5.375000) | (2.3256%)
( 04 )      | (5.250000) | (5.375000) | (5.312500) | (1.1765%)
( 05 )      | (5.312500) | (5.375000) | (5.343750) | (0.5848%)
( 06 )      | (5.312500) | (5.343750) | (5.328125) | (0.2933%)
( 07 )      | (5.312500) | (5.328125) | (5.320312) | (0.1468%)
( 08 )      | (5.320312) | (5.328125) | (5.324219) | (0.0734%)
( 09 )      | (5.320312) | (5.324219) | (5.322266) | (0.0367%)
( 10 )      | (5.322266) | (5.324219) | (5.323242) | (0.0183%)
( 11 )      | (5.322266) | (5.323242) | (5.322754) | (0.0092%)
( 12 )      | (5.322266) | (5.322754) | (5.322510) | (0.0046%)
( 13 )      | (5.322266) | (5.322510) | (5.322388) | (0.0023%)
( 14 )      | (5.322266) | (5.322388) | (5.322327) | (0.0011%)
( 15 )      | (5.322266) | (5.322327) | (5.322296) | (0.0006%)
-------------------------------------------------------------------
Raiz encontrada: 5.322296 com erro de 0.0006% apos 15 iteracoes.
```

Saída – Falsa Posição (Entrada 2)

```
====== RELATORIO ======
Funcao analisada: f(x) = 0.1 - (1 + x + (x**2)/2) * math.exp(-x)
Intervalo inicial: [5.0, 6.0]
Tolerancia exigida: 0.001%
=======================

METODO DA FALSA POSICAO
(Iteracao) | (xl)       | (xu)       | (xr)       | (ea %)
-----------------------------------------------------------------
( 01 )     | (5.000000) | (6.000000) | (5.393279) | (100.0000%)
( 02 )     | (5.000000) | (5.393279) | (5.329199) | (1.2024%)
( 03 )     | (5.000000) | (5.329199) | (5.322982) | (0.1168%)
( 04 )     | (5.000000) | (5.322982) | (5.322384) | (0.0112%)
( 05 )     | (5.000000) | (5.322384) | (5.322326) | (0.0011%)
( 06 )     | (5.000000) | (5.322326) | (5.322321) | (0.0001%)
-----------------------------------------------------------------
Raiz encontrada: 5.322321 com erro de 0.0001% apos 6 iteracoes.
```

Saída – Newton-Raphson (Entrada 2)

```
====== RELATORIO DE ANALISE NUMERICA ======
Funcao original : f(x)  = 0.1 - (1 + x + (x**2)/2) * math.exp(-x)
Derivada        : f'(x) = ((x**2) / 2) * math.exp(-x)
Chute inicial   : x0 = 5.5
Tolerancia (ea) : 0.001%
==========================================

METODO DE NEWTON-RAPHSON
(Iteracao) | (x_atual)  | (f(x))     | (df(x))    | (ea %)
-----------------------------------------------------------------
( 01 )     | (5.500000) | (0.011624) | (0.061812) | (3.540050%)
( 02 )     | (5.311954) | (-0.000719) | (0.069587) | (0.194141%)
( 03 )     | (5.322287) | (-0.000002) | (0.069140) | (0.000628%)
( 04 )     | (5.322320) | (-0.000000) | (0.069138) | (0.000628%)
-----------------------------------------------------------------
Raiz encontrada: 5.32232034 com erro de 0.000628% apos 3 iteracoes.
```

Saída – Secante (Entrada 2)

```
====== RELATORIO DE ANALISE NUMERICA ======
Funcao analisada : f(x) = 0.1 - (1 + x + (x**2)/2) * math.exp(-x)
Chutes iniciais  : x0 = 5.0, x1 = 6.0
Tolerancia (ea)  : 0.001%
==========================================

METODO DA SECANTE
(Iteracao) | (x_ant)    | (x_atual)  | (x_novo)   | (ea %)
---------------------------------------------------------------
( 01 )     | (5.000000) | (6.000000) | (5.393279) | (11.249567%)
( 02 )     | (6.000000) | (5.393279) | (5.305672) | (1.651209%)
( 03 )     | (5.393279) | (5.305672) | (5.322692) | (0.319766%)
( 04 )     | (5.305672) | (5.322692) | (5.322322) | (0.006945%)
( 05 )     | (5.322692) | (5.322322) | (5.322320) | (0.000036%)
---------------------------------------------------------------
Raiz encontrada: 5.32232034 com erro de 0.000036% apos 5 iteracoes.
```

$T_{0,1}$ = 1,10

$T_{0,9}$ = 5,32

5,32 – 1,10 = 4,22

Resposta 3.3: O Tempo de subida é de aproximadamente 4,22 u.t (unidades de tempo).

Problema Teste 2 - Página 100 Questão 3.6

3.6 - *A equação:*

$$tg\left(\frac{\theta}{2}\right) = \frac{sen\ \alpha\ cos\ \alpha}{\frac{g\ R}{v^2} - cos^2\ \alpha}\ ,$$

permite calcular o ângulo de inclinação, α, em que o lançamento do míssil deve ser feito para atingir um determinado alvo. Na equação acima,

- α *- ângulo de inclinação com a superfície da Terra com a qual é feita o lançamento do míssil* ,
- g *- aceleração da gravidade* $\simeq$ $9.81\ m/s^2$,
- R *- raio da Terra* $\simeq$ $6371000\ m$,
- v *- velocidade de lançamento do míssil,* m/s ,
- θ *- ângulo (medido do centro da Terra) entre o ponto de lançamento e o ponto de impacto desejado* ,

Resolva o problema considerando: $\theta = 80^o$ *e* v *tal que* $\frac{v^2}{gR} = 1.25$, *ou seja, aproximadamente* $8.840\ m/s$.

Entrada 1 – para Bissecção, Falsa Posição e Secante

```
(math.sin(x) * math.cos(x)) / (0.8 - (math.cos(x))**2) - math.tan(math.radians(40))
0.9
1.2
0.01
```

Entrada 1 – para Newton-Raphson

```
(math.sin(x) * math.cos(x)) / (0.8 - (math.cos(x))**2) - math.tan(math.radians(40))
(((math.cos(x)**2 - math.sin(x)**2) * (0.8 - math.cos(x)**2)) - ((math.sin(x) * math.cos(x)) * (2 * math.sin(x) *
math.cos(x)))) / (0.8 - math.cos(x)**2)**2
1.0
0.01
```

Saída – Bissecção (Entrada 1)

```
====== RELATORIO ======
Funcao analisada: f(x) = (math.sin(x) * math.cos(x)) / (0.8 - (math.cos(x))**2) - math.tan(math.radians(40))
Intervalo inicial: [0.9, 1.2]
Tolerancia exigida: 0.01%
======================

METODO DA BISSECCAO
(Iteracao) | (xl)       | (xu)       | (xr)       | (ea %)
-----------------------------------------------------------------
( 01 )     | (0.900000) | (1.200000) | (1.050000) | (14.2857%)
( 02 )     | (0.900000) | (1.050000) | (0.975000) | (7.6923%)
( 03 )     | (0.975000) | (1.050000) | (1.012500) | (3.7037%)
( 04 )     | (1.012500) | (1.050000) | (1.031250) | (1.8182%)
( 05 )     | (1.012500) | (1.031250) | (1.021875) | (0.9174%)
( 06 )     | (1.021875) | (1.031250) | (1.026563) | (0.4566%)
( 07 )     | (1.021875) | (1.026563) | (1.024219) | (0.2288%)
( 08 )     | (1.021875) | (1.024219) | (1.023047) | (0.1145%)
( 09 )     | (1.023047) | (1.024219) | (1.023633) | (0.0572%)
( 10 )     | (1.023633) | (1.024219) | (1.023926) | (0.0286%)
( 11 )     | (1.023633) | (1.023926) | (1.023779) | (0.0143%)
( 12 )     | (1.023633) | (1.023779) | (1.023706) | (0.0072%)
-----------------------------------------------------------------
Raiz encontrada: 1.023706 com erro de 0.0072% apos 12 iteracoes.
```

Saída – Falsa Posição (Entrada 1)

```
====== RELATORIO ======
Funcao analisada: f(x) = (math.sin(x) * math.cos(x)) / (0.8 - (math.cos(x))**2) - math.tan(math.radians(40))
Intervalo inicial: [0.9, 1.2]
Tolerancia exigida: 0.01%
=======================

METODO DA FALSA POSICAO
(Iteracao) | (xl)       | (xu)       | (xr)       | (ea %)
-----------------------------------------------------------------
( 01 )      | (0.900000) | (1.200000) | (1.050924) | (100.0000%)
( 02 )      | (0.900000) | (1.050924) | (1.028254) | (2.2047%)
( 03 )      | (0.900000) | (1.028254) | (1.024513) | (0.3652%)
( 04 )      | (0.900000) | (1.024513) | (1.023888) | (0.0610%)
( 05 )      | (0.900000) | (1.023888) | (1.023783) | (0.0102%)
( 06 )      | (0.900000) | (1.023783) | (1.023766) | (0.0017%)
-----------------------------------------------------------------
Raiz encontrada: 1.023766 com erro de 0.0017% apos 6 iteracoes.
```

Saída – Newton-Raphson (Entrada 1)

```
====== RELATORIO DE ANALISE NUMERICA ======
Funcao original : f(x)  = (math.sin(x) * math.cos(x)) / (0.8 - (math.cos(x))**2) - math.tan(math.radians(40))
Derivada        : f'(x) = (((math.cos(x)**2 - math.sin(x)**2) * (0.8 - math.cos(x)**2)) - ((math.sin(x) * math.cos
(x)) * (2 * math.sin(x) * math.cos(x)))) / (0.8 - math.cos(x)**2)**2
Chute inicial   : x0 = 1.0
Tolerancia (ea) : 0.01%
==========================================

METODO DE NEWTON-RAPHSON
(Iteracao) | (x_atual)  | (f(x))     | (df(x))    | (ea %)
-----------------------------------------------------------------
( 01 )      | (1.000000) | (0.055749) | (-2.420576) | (2.251273%)
( 02 )      | (1.023031) | (0.001664) | (-2.279097) | (0.071329%)
( 03 )      | (1.023761) | (0.000002) | (-2.274901) | (0.000066%)
( 04 )      | (1.023762) | (0.000000) | (-2.274898) | (0.000066%)
-----------------------------------------------------------------
Raiz encontrada: 1.02376214 com erro de 0.000066% apos 3 iteracoes.
```

Saída – Secante (Entrada 1)

```
====== RELATORIO DE ANALISE NUMERICA ======
Funcao analisada : f(x) = (math.sin(x) * math.cos(x)) / (0.8 - (math.cos(x))**2) - math.tan(math.radians(40))
Chutes iniciais  : x0 = 0.9, x1 = 1.2
Tolerancia (ea)  : 0.01%
==========================================

METODO DA SECANTE
(Iteracao) | (x_ant)    | (x_atual)  | (x_novo)   | (ea %)
-----------------------------------------------------------------
( 01 )      | (0.900000) | (1.200000) | (1.050924) | (14.185276%)
( 02 )      | (1.200000) | (1.050924) | (1.018433) | (3.190278%)
( 03 )      | (1.050924) | (1.018433) | (1.023942) | (0.538071%)
( 04 )      | (1.018433) | (1.023942) | (1.023763) | (0.017479%)
( 05 )      | (1.023942) | (1.023763) | (1.023762) | (0.000119%)
-----------------------------------------------------------------
Raiz encontrada: 1.02376214 com erro de 0.000119% apos 5 iteracoes.
```

α = 1,023 rad

$$1,023 \times \frac{180°}{\pi} = 58,613°$$

Resposta 3.6: O ângulo de inclinação em que o lançamento do míssil deve ser feito para atingir o determinado alvo é de aproximadamente 58,613°.

Problema Teste 3 - Página 100 Questão 3.8

3.8 - *Uma loja de eletrodomésticos oferece dois planos de financiamento para um produto cujo preço a vista é R$ 162,00:*

- *Plano A: entrada de R$ 22,00 + 9 prestações iguais de R$ 26,50,*
- *Plano B: entrada de R$ 22,00 + 12 prestações de R$ 21,50.*

Qual dos dois planos apresenta a menor taxa de juros, sendo portanto melhor para o consumidor?

Observação: *Sabe-se que a equação que relaciona os juros* (**J**) *e o prazo* (**P**) *com o valor financiado* (**VF** = *preço à vista - entrada*) *e a prestação mensal* **PM** *é dada por:*

$$\frac{1-(1+J)^{-P}}{J} = \frac{VF}{PM} \ . \tag{3.33}$$

a) *Fazendo* $x = 1 + J$ *e* $k = \frac{VF}{PM}$, *verificar que a equação* (3.33) *se transforma em:*

$$f(x) = kx^{P+1} - (k+1)x^P + 1 = 0 \ . \tag{3.34}$$

b) *Escrever a equação* (3.34) *para o problema proposto e encontrar um intervalo contendo a raiz positiva* $\neq 1$.

Entrada 1 – para Bissecção, Falsa Posição e Secante

```
5.2830 * (x**10) - 6.2830 * (x**9) + 1
1.1
1.2
0.001
```

Entrada 1 – para Newton-Raphson

```
5.2830 * (x**10) - 6.2830 * (x**9) + 1
52.83 * (x**9) - 56.547 * (x**8)
1.15
0.001
```

Entrada 2 – para Bissecção, Falsa Posição e Secante

```
6.5116 * (x**13) - 7.5116 * (x**12) + 1
1.1
1.2
0.001
```

Entrada 2 – para Newton-Raphson

```
6.5116 * (x**13) - 7.5116 * (x**12) + 1
84.6508 * (x**12) - 90.1392 * (x**11)
1.15
0.001
```

Saída – Bissecção (Entrada 1) – Plano A

```
====== RELATORIO ======
Funcao analisada: f(x) = 5.2830 * (x**10) - 6.2830 * (x**9) + 1
Intervalo inicial: [1.1, 1.2]
Tolerancia exigida: 0.001%
=======================

METODO DA BISSECCAO
(Iteracao) | (xl)       | (xu)       | (xr)       | (ea %)
-----------------------------------------------------------------
( 01 )      | (1.100000) | (1.200000) | (1.150000) | (4.3478%)
( 02 )      | (1.100000) | (1.150000) | (1.125000) | (2.2222%)
( 03 )      | (1.100000) | (1.125000) | (1.112500) | (1.1236%)
( 04 )      | (1.112500) | (1.125000) | (1.118750) | (0.5587%)
( 05 )      | (1.118750) | (1.125000) | (1.121875) | (0.2786%)
( 06 )      | (1.121875) | (1.125000) | (1.123438) | (0.1391%)
( 07 )      | (1.121875) | (1.123438) | (1.122656) | (0.0696%)
( 08 )      | (1.121875) | (1.122656) | (1.122266) | (0.0348%)
( 09 )      | (1.121875) | (1.122266) | (1.122070) | (0.0174%)
( 10 )      | (1.122070) | (1.122266) | (1.122168) | (0.0087%)
( 11 )      | (1.122168) | (1.122266) | (1.122217) | (0.0044%)
( 12 )      | (1.122217) | (1.122266) | (1.122241) | (0.0022%)
( 13 )      | (1.122241) | (1.122266) | (1.122253) | (0.0011%)
( 14 )      | (1.122241) | (1.122253) | (1.122247) | (0.0005%)
-----------------------------------------------------------------
Raiz encontrada: 1.122247 com erro de 0.0005% apos 14 iteracoes.
```

Saída – Falsa Posição (Entrada 1) – Plano A

```
====== RELATORIO ======
Funcao analisada: f(x) = 5.2830 * (x**10) - 6.2830 * (x**9) + 1
Intervalo inicial: [1.1, 1.2]
Tolerancia exigida: 0.001%
=======================

METODO DA FALSA POSICAO
(Iteracao) | (xl)       | (xu)       | (xr)       | (ea %)
-----------------------------------------------------------------
( 01 )     | (1.100000) | (1.200000) | (1.107993) | (100.0000%)
( 02 )     | (1.107993) | (1.200000) | (1.113412) | (0.4867%)
( 03 )     | (1.113412) | (1.200000) | (1.116891) | (0.3115%)
( 04 )     | (1.116891) | (1.200000) | (1.119047) | (0.1926%)
( 05 )     | (1.119047) | (1.200000) | (1.120352) | (0.1165%)
( 06 )     | (1.120352) | (1.200000) | (1.121131) | (0.0695%)
( 07 )     | (1.121131) | (1.200000) | (1.121592) | (0.0411%)
( 08 )     | (1.121592) | (1.200000) | (1.121864) | (0.0242%)
( 09 )     | (1.121864) | (1.200000) | (1.122023) | (0.0142%)
( 10 )     | (1.122023) | (1.200000) | (1.122117) | (0.0083%)
( 11 )     | (1.122117) | (1.200000) | (1.122172) | (0.0049%)
( 12 )     | (1.122172) | (1.200000) | (1.122204) | (0.0029%)
( 13 )     | (1.122204) | (1.200000) | (1.122223) | (0.0017%)
( 14 )     | (1.122223) | (1.200000) | (1.122234) | (0.0010%)
-----------------------------------------------------------------
Raiz encontrada: 1.122234 com erro de 0.0010% apos 14 iteracoes.
```

Saída – Newton-Raphson (Entrada 1) – Plano A

```
====== RELATORIO DE ANALISE NUMERICA ======
Funcao original : f(x)  = 5.2830 * (x**10) - 6.2830 * (x**9) + 1
Derivada        : f'(x) = 52.83 * (x**9) - 56.547 * (x**8)
Chute inicial   : x0 = 1.15
Tolerancia (ea) : 0.001%
==========================================

METODO DE NEWTON-RAPHSON
(Iteracao) | (x_atual)  | (f(x))     | (df(x))    | (ea %)
-----------------------------------------------------------------
( 01 )     | (1.150000) | (0.269865) | (12.870839) | (1.857089%)
( 02 )     | (1.129033) | (0.051096) | (8.184405) | (0.556032%)
( 03 )     | (1.122790) | (0.003755) | (6.996244) | (0.047825%)
( 04 )     | (1.122253) | (0.000026) | (6.898191) | (0.000340%)
( 05 )     | (1.122249) | (0.000000) | (6.897496) | (0.000340%)
-----------------------------------------------------------------
Raiz encontrada: 1.12224925 com erro de 0.000340% apos 4 iteracoes.
```

Saída – Secante (Entrada 1) – Plano A

```
====== RELATORIO DE ANALISE NUMERICA ======
Funcao analisada : f(x) = 5.2830 * (x**10) - 6.2830 * (x**9) + 1
Chutes iniciais  : x0 = 1.1, x1 = 1.2
Tolerancia (ea)  : 0.001%
==========================================

METODO DA SECANTE
(Iteracao) | (x_ant)    | (x_atual)  | (x_novo)   | (ea %)
-----------------------------------------------------------------
( 01 )     | (1.100000) | (1.200000) | (1.107993) | (8.303939%)
( 02 )     | (1.200000) | (1.107993) | (1.113412) | (0.486692%)
( 03 )     | (1.107993) | (1.113412) | (1.124364) | (0.974045%)
( 04 )     | (1.113412) | (1.124364) | (1.121987) | (0.211849%)
( 05 )     | (1.124364) | (1.121987) | (1.122242) | (0.022751%)
( 06 )     | (1.121987) | (1.122242) | (1.122249) | (0.000644%)
-----------------------------------------------------------------
Raiz encontrada: 1.12224928 com erro de 0.000644% apos 6 iteracoes.
```

Saída – Bissecção (Entrada 2) – Plano B

```
====== RELATORIO ======
Funcao analisada: f(x) = 6.5116 * (x**13) - 7.5116 * (x**12) + 1
Intervalo inicial: [1.1, 1.2]
Tolerancia exigida: 0.001%
=======================

METODO DA BISSECCAO
(Iteracao) | (xl)       | (xu)       | (xr)       | (ea %)
-----------------------------------------------------------------
( 01 )     | (1.100000) | (1.200000) | (1.150000) | (4.3478%)
( 02 )     | (1.100000) | (1.150000) | (1.125000) | (2.2222%)
( 03 )     | (1.100000) | (1.125000) | (1.112500) | (1.1236%)
( 04 )     | (1.100000) | (1.112500) | (1.106250) | (0.5650%)
( 05 )     | (1.106250) | (1.112500) | (1.109375) | (0.2817%)
( 06 )     | (1.109375) | (1.112500) | (1.110937) | (0.1406%)
( 07 )     | (1.109375) | (1.110937) | (1.110156) | (0.0704%)
( 08 )     | (1.109375) | (1.110156) | (1.109766) | (0.0352%)
( 09 )     | (1.109375) | (1.109766) | (1.109570) | (0.0176%)
( 10 )     | (1.109375) | (1.109570) | (1.109473) | (0.0088%)
( 11 )     | (1.109375) | (1.109473) | (1.109424) | (0.0044%)
( 12 )     | (1.109375) | (1.109424) | (1.109399) | (0.0022%)
( 13 )     | (1.109375) | (1.109399) | (1.109387) | (0.0011%)
( 14 )     | (1.109375) | (1.109387) | (1.109381) | (0.0006%)
-----------------------------------------------------------------
Raiz encontrada: 1.109381 com erro de 0.0006% apos 14 iteracoes.
```

Saída – Falsa Posição (Entrada 2) – Plano B

```
====== RELATORIO ======
Funcao analisada: f(x) = 6.5116 * (x**13) - 7.5116 * (x**12) + 1
Intervalo inicial: [1.1, 1.2]
Tolerancia exigida: 0.001%
=======================

METODO DA FALSA POSICAO
(Iteracao) | (xl)       | (xu)       | (xr)       | (ea %)
-----------------------------------------------------------------
( 01 )      | (1.100000) | (1.200000) | (1.102501) | (100.0000%)
( 02 )      | (1.102501) | (1.200000) | (1.104378) | (0.1699%)
( 03 )      | (1.104378) | (1.200000) | (1.105766) | (0.1255%)
( 04 )      | (1.105766) | (1.200000) | (1.106781) | (0.0917%)
( 05 )      | (1.106781) | (1.200000) | (1.107517) | (0.0665%)
( 06 )      | (1.107517) | (1.200000) | (1.108048) | (0.0479%)
( 07 )      | (1.108048) | (1.200000) | (1.108429) | (0.0344%)
( 08 )      | (1.108429) | (1.200000) | (1.108702) | (0.0246%)
( 09 )      | (1.108702) | (1.200000) | (1.108897) | (0.0176%)
( 10 )      | (1.108897) | (1.200000) | (1.109036) | (0.0125%)
( 11 )      | (1.109036) | (1.200000) | (1.109135) | (0.0089%)
( 12 )      | (1.109135) | (1.200000) | (1.109206) | (0.0064%)
( 13 )      | (1.109206) | (1.200000) | (1.109256) | (0.0045%)
( 14 )      | (1.109256) | (1.200000) | (1.109292) | (0.0032%)
( 15 )      | (1.109292) | (1.200000) | (1.109317) | (0.0023%)
( 16 )      | (1.109317) | (1.200000) | (1.109335) | (0.0016%)
( 17 )      | (1.109335) | (1.200000) | (1.109348) | (0.0012%)
( 18 )      | (1.109348) | (1.200000) | (1.109357) | (0.0008%)
-----------------------------------------------------------------
Raiz encontrada: 1.109357 com erro de 0.0008% apos 18 iteracoes.
```

Saída – Newton-Raphson (Entrada 2) – Plano B

```
====== RELATORIO DE ANALISE NUMERICA ======
Funcao original : f(x)  = 6.5116 * (x**13) - 7.5116 * (x**12) + 1
Derivada        : f'(x) = 84.6508 * (x**12) - 90.1392 * (x**11)
Chute inicial   : x0 = 1.15
Tolerancia (ea) : 0.001%
==========================================

METODO DE NEWTON-RAPHSON
(Iteracao) | (x_atual)  | (f(x))     | (df(x))    | (ea %)
-----------------------------------------------------------------
( 01 )      | (1.150000) | (0.875553) | (33.540113) | (2.322695%)
( 02 )      | (1.123895) | (0.215100) | (18.067827) | (1.070615%)
( 03 )      | (1.111990) | (0.032160) | (12.831111) | (0.225911%)
( 04 )      | (1.109484) | (0.001236) | (11.851232) | (0.009404%)
( 05 )      | (1.109379) | (0.000002) | (11.811315) | (0.000016%)
( 06 )      | (1.109379) | (0.000000) | (11.811247) | (0.000016%)
-----------------------------------------------------------------
Raiz encontrada: 1.10937926 com erro de 0.000016% apos 5 iteracoes.
```

Saída – Secante (Entrada 2) – Plano B

```
====== RELATORIO DE ANALISE NUMERICA ======
Funcao analisada : f(x) = 6.5116 * (x**13) - 7.5116 * (x**12) + 1
Chutes iniciais  : x0 = 1.1, x1 = 1.2
Tolerancia (ea)  : 0.001%
==========================================

METODO DA SECANTE
(Iteracao) | (x_ant)    | (x_atual)  | (x_novo)   | (ea %)
-----------------------------------------------------------------
( 01 )      | (1.100000) | (1.200000) | (1.102501) | (8.843404%)
( 02 )      | (1.200000) | (1.102501) | (1.104378) | (0.169934%)
( 03 )      | (1.102501) | (1.104378) | (1.110017) | (0.508026%)
( 04 )      | (1.104378) | (1.110017) | (1.109325) | (0.062391%)
( 05 )      | (1.110017) | (1.109325) | (1.109379) | (0.004829%)
( 06 )      | (1.109325) | (1.109379) | (1.109379) | (0.000050%)
-----------------------------------------------------------------
Raiz encontrada: 1.10937926 com erro de 0.000050% apos 6 iteracoes.
```

Resposta 3.8: O plano com a menor taxa de juros é o plano B, com aproximadamente 10,93% de taxa, enquanto o Plano A tem aproximadamente 12,22% de taxa.

Método da Eliminação de Gauss

Estratégia de Implementação:

A Eliminação de Gauss foi implementada em sua forma estrita e direta, conhecida como "Gauss Ingênuo" (sem pivoteamento). O algoritmo é dividido em duas grandes fases. Na Fase 1 (Eliminação Progressiva), o programa zera iterativamente os elementos abaixo da diagonal principal. Um cuidado fundamental no código foi forçar a atribuição de zero (A[i][k] = 0.0) nas posições eliminadas, o que previne o acúmulo de valores residuais indesejados gerados por aproximações de ponto flutuante. Além disso, introduziu-se uma trava de segurança essencial: antes de calcular o fator multiplicador, o programa verifica se o pivô (A[k][k]) é igual a zero. Caso seja, o sistema emite um "ERRO FATAL" e aborta a operação para impedir uma quebra por divisão por zero. A Fase 2 (Substituição Regressiva) simplesmente percorre o sistema triangular superior recém-criado de baixo para cima, isolando cada variável de forma sucessiva.

Estrutura dos Arquivos de Entrada/Saída

O arquivo "entrada.txt" segue um formato de matriz aumentada altamente prático. A primeira linha dita a dimensão do sistema (n), enquanto as linhas subsequentes contêm os coeficientes da matriz A seguidos imediatamente pelo termo independente b correspondente àquela linha. Essa escolha simplificou a rotina de leitura (usando um fatiamento de listas no Python). No "saida.txt", optou-se por documentar o passo a passo das operações elementares aplicadas sobre as linhas (exibindo qual multiplicador foi usado). Ao final, o vetor de solução é impresso formatado entre parênteses, entregando o resultado final de maneira clara e padronizada.

Dificuldades enfrentadas

A maior limitação teórica do Método de Gauss Ingênuo é justamente a ausência de pivoteamento parcial. Se o algoritmo se deparar com um zero exato na diagonal principal durante o processamento, ele falha imediatamente e não consegue prosseguir, mesmo que o sistema tenha uma solução válida

(situação que seria resolvida com uma simples troca de linhas). Outro grande desafio desse método é a sensibilidade ao erro de arredondamento: se um pivô for um número não nulo, mas muito próximo de zero, o fator multiplicador se tornará gigantesco, amplificando os erros de precisão nas equações inferiores e corrompendo a exatidão das raízes encontradas.

Método da Fatoração LU

Estratégia de Implementação:

O código executa a decomposição do sistema original separando a matriz de coeficientes A em uma matriz triangular inferior L (com 1s preenchendo a diagonal principal) e uma matriz triangular superior U. A arquitetura foi dividida em três fases lógicas.

Na Fase 1, os laços aninhados preenchem primeiramente as linhas da matriz U e, em seguida, as colunas da matriz L. Assim como no método de Gauss, manteve-se a checagem rigorosa contra pivôs nulos (U[i][i] == 0.0) antes de realizar a divisão para compor os fatores de L.

Na Fase 2 (Substituição Progressiva), o algoritmo resolve o sistema intermediário L * y = b de cima para baixo. Uma vantagem matemática percebida nesta etapa é que, como a diagonal principal de L possui apenas o número 1, o código encontra o vetor 'y' executando álgebra pura (apenas multiplicações e subtrações), sem a necessidade de divisões.

A Fase 3 (Substituição Regressiva) espelha a lógica do método de Gauss, resolvendo U * x = y de baixo para cima para encontrar as respostas definitivas.

Estrutura dos Arquivos de Entrada/Saída

A interface de entrada ("entrada.txt") foi estrategicamente mantida idêntica à da Eliminação de Gauss (recebendo a dimensão n e a matriz aumentada A conjunta com b). A justificativa para essa padronização é puramente prática: isso confere flexibilidade ao laboratório de testes, permitindo alimentar o mesmo problema numérico em ambos os programas sem precisar reformatar o arquivo de texto. Já o "saida.txt" foi formatado para ser visualmente auditável; ele imprime por completo as novas matrizes L e U geradas, além de detalhar os valores do vetor intermediário (y) e, por fim, o vetor solução (x) encapsulado por parênteses.

Dificuldades enfrentadas

A complexidade primária na implementação da Fatoração LU está no aninhamento triplo dos laços de repetição e no rigor exigido pelos índices de iteração para calcular as somatórias parciais. Um simples desvio de índice faria os valores serem alocados de forma cruzada, quebrando a identidade das matrizes. Do ponto de vista numérico, assim como o Gauss Ingênuo, o código sofre com a limitação de não realizar trocas de linhas, sendo vulnerável a pivôs zerados. No entanto, o benefício de longo prazo compensa essa complexidade: uma vez que as matrizes L e U estão prontas, torna-se possível calcular as raízes para variados vetores 'b' diferentes utilizando apenas as rotinas de substituição das Fases 2 e 3, exigindo um poder de processamento massivamente inferior ao de recalcular o sistema do zero.

Problema Teste 1 - Página 150 Questão 4.1

4.1 - *Considere o circuito a seguir com resistências e baterias tal como indicado; escolhemos arbitrariamente as correntes e os valores da malha:*

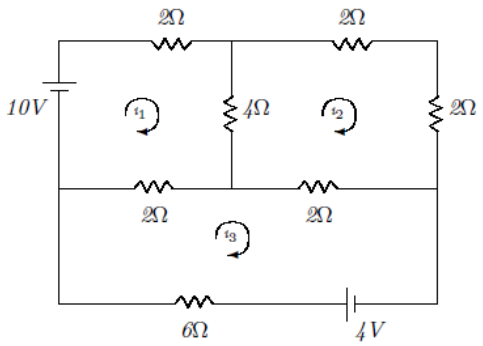


Figura 4.4

Aplicando a Lei de Kirchoff que diz que a soma algébrica da diferenças de potencial em qualquer circuito fechado é zero, obtemos para as correntes i_1, i_2, i_3, o seguinte sistema linear:

$$\begin{cases} 2\,i_1 + 4\,(i_1 - i_2) + 2\,(i_1 - i_3) - 10 = 0 \\ 2\,i_2 - 2\,i_2 + 2\,(i_2 - i_3) + 4\,(i_2 - i_1) = 0 \\ 6\,i_3 + 2\,(i_3 - i_1) + 2\,(i_3 - i_2) - 4 = 0 \end{cases}$$

Deseja-se determinar o valor de $i = (i_1,\ i_2,\ i_3)^t$ que satisfaça o sistema acima.

a) *É possível resolver o sistema pelo método da decomposição LU? Justifique.*

b) *É possível resolver o sistema pelo método de Cholesky? Justifique.*

c) *Resolva o sistema pelo método de Eliminação de Gauss.*

Entrada 1 – para Eliminação de Gauss e Fatoração LU

```
3
8 -4 -2 10
-4 6 -2 0
-2 -2 10 4
```

Saída – Eliminação de Gauss (Entrada 1)

```
====== RELATORIO DE SISTEMAS LINEARES ======
Tamanho do sistema: 3x3
==========================================

--- FASE 1: ELIMINACAO PROGRESSIVA ---

Eliminando variaveis abaixo do pivo A(1, 1) = 8.0:
Linha 2 = Linha 2 - (-0.5000) * Linha 1
Linha 3 = Linha 3 - (-0.2500) * Linha 1

Eliminando variaveis abaixo do pivo A(2, 2) = 4.0:
Linha 3 = Linha 3 - (-0.7500) * Linha 2

--- FASE 2: SUBSTITUICAO REGRESSIVA ---
x(3) = 1.4138
x(2) = 2.3103
x(1) = 2.7586

------------------------------------------
SOLUCAO FINAL ENCONTRADA:
Vetor x = (2.7586, 2.3103, 1.4138)
------------------------------------------
```

Saída – Fatoração LU (Entrada 1)

```
====== RELATORIO DE SISTEMAS LINEARES ======
Metodo: Fatoracao LU
Tamanho do sistema: 3x3
==========================================

--- FASE 1: FATORACAO A = L * U ---

Matriz [L] (Inferior com os fatores):
    1.0000    0.0000    0.0000
   -0.5000    1.0000    0.0000
   -0.2500   -0.7500    1.0000

Matriz [U] (Superior):
    8.0000   -4.0000   -2.0000
    0.0000    4.0000   -3.0000
    0.0000    0.0000    7.2500

--- FASE 2: SUBSTITUICAO PROGRESSIVA (L * y = b) ---
y(1) = 10.0000
y(2) = 5.0000
y(3) = 10.2500

--- FASE 3: SUBSTITUICAO REGRESSIVA (U * x = y) ---
x(3) = 1.4138
x(2) = 2.3103
x(1) = 2.7586

------------------------------------------
SOLUCAO FINAL ENCONTRADA:
Vetor x = (2.7586, 2.3103, 1.4138)
------------------------------------------
```

Resposta 4.1:

i_1 = 2,7586 A

i_2 = 2,3103 A

i_3 = 1,4138 A

Problema Teste 2 - Página 151 Questão 4.3

4.3 - *Um cachorro está perdido em um labirinto quadrado de corredores (Figura 4.5). Em cada interseção escolhe uma direção ao acaso e segue até a interseção seguinte onde escolhe novamente ao acaso nova direção e assim por diante. Qual a probabilidade do cachorro, estando na interseção i, sair eventualmente pelo lado sul?*

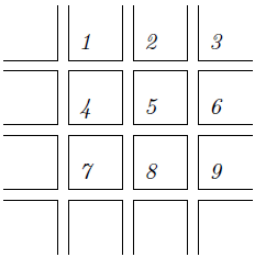


Figura 4.5

Esclarecimentos: *Suponhamos que há exatamente as 9 interseções mostradas na figura. Seja P_1 a probabilidade do cachorro, que está na interseção 1, sair pelo lado sul. Seja $P_2, P_3, \ldots, P_9$ definidas de modo similar. Supondo que em cada interseção a que chegue o cachorro, há tanta possibilidade que escolha uma direção como outra, e que, tendo chegado a uma saída, tenha terminado sua caminhada, a teoria das probabilidades oferece as seguintes equações para P_i:*

$$\begin{cases} P_1 &= (0+0+P_2+P_4)/4 \\ P_2 &= (0+P_1+P_3+P_5)/4 \\ P_3 &= (0+P_2+0+P_6)/4 \\ P_4 &= (P_1+0+P_5+P_7)/4 \\ P_5 &= (P_2+P_4+P_6+P_8)/4 \\ P_6 &= (P_3+P_5+0+P_9)/4 \\ P_7 &= (P_4+0+P_8+1)/4 \\ P_8 &= (P_5+P_7+P_9+1)/4 \\ P_9 &= (P_6+P_8+0+1)/4 \end{cases}$$

Para saber a resposta resolva o sistema linear obtido.

Entrada 1 – para Eliminação de Gauss e Fatoração LU

```
9
4 -1 0 -1 0 0 0 0 0 0
-1 4 -1 0 -1 0 0 0 0 0
0 -1 4 0 0 -1 0 0 0 0
-1 0 0 4 -1 0 -1 0 0 0
0 -1 0 -1 4 -1 0 -1 0 0
0 0 -1 0 -1 4 0 0 -1 0
0 0 0 -1 0 0 4 -1 0 1
0 0 0 0 -1 0 -1 4 -1 1
0 0 0 0 0 -1 0 -1 4 1
```

Saída – Eliminação de Gauss (Entrada 1)

```
====== RELATORIO DE SISTEMAS LINEARES ======
Tamanho do sistema: 9x9
==========================================

--- FASE 1: ELIMINACAO PROGRESSIVA ---

Eliminando variaveis abaixo do pivo A(1, 1) = 4.0:
Linha 2 = Linha 2 - (-0.2500) * Linha 1
Linha 3 = Linha 3 - (0.0000) * Linha 1
Linha 4 = Linha 4 - (-0.2500) * Linha 1
Linha 5 = Linha 5 - (0.0000) * Linha 1
Linha 6 = Linha 6 - (0.0000) * Linha 1
Linha 7 = Linha 7 - (0.0000) * Linha 1
Linha 8 = Linha 8 - (0.0000) * Linha 1
Linha 9 = Linha 9 - (0.0000) * Linha 1

Eliminando variaveis abaixo do pivo A(2, 2) = 3.75:
Linha 3 = Linha 3 - (-0.2667) * Linha 2
Linha 4 = Linha 4 - (-0.0667) * Linha 2
Linha 5 = Linha 5 - (-0.2667) * Linha 2
Linha 6 = Linha 6 - (0.0000) * Linha 2
Linha 7 = Linha 7 - (0.0000) * Linha 2
Linha 8 = Linha 8 - (0.0000) * Linha 2
Linha 9 = Linha 9 - (0.0000) * Linha 2

Eliminando variaveis abaixo do pivo A(3, 3) = 3.7333333333333334:
Linha 4 = Linha 4 - (-0.0179) * Linha 3
Linha 5 = Linha 5 - (-0.0714) * Linha 3
Linha 6 = Linha 6 - (-0.2679) * Linha 3
Linha 7 = Linha 7 - (0.0000) * Linha 3
Linha 8 = Linha 8 - (0.0000) * Linha 3
Linha 9 = Linha 9 - (0.0000) * Linha 3

Eliminando variaveis abaixo do pivo A(4, 4) = 3.732142857142857:
Linha 5 = Linha 5 - (-0.2871) * Linha 4
Linha 6 = Linha 6 - (-0.0048) * Linha 4
Linha 7 = Linha 7 - (-0.2679) * Linha 4
Linha 8 = Linha 8 - (0.0000) * Linha 4
Linha 9 = Linha 9 - (0.0000) * Linha 4

Eliminando variaveis abaixo do pivo A(5, 5) = 3.4066985645933014:
Linha 6 = Linha 6 - (-0.3160) * Linha 5
Linha 7 = Linha 7 - (-0.0843) * Linha 5
Linha 8 = Linha 8 - (-0.2935) * Linha 5
Linha 9 = Linha 9 - (0.0000) * Linha 5

Eliminando variaveis abaixo do pivo A(6, 6) = 3.3918539325842696:
Linha 7 = Linha 7 - (-0.0282) * Linha 6
Linha 8 = Linha 8 - (-0.0932) * Linha 6
Linha 9 = Linha 9 - (-0.2948) * Linha 6

Eliminando variaveis abaixo do pivo A(7, 7) = 3.705175983436853:
Linha 8 = Linha 8 - (-0.2950) * Linha 7
Linha 9 = Linha 9 - (-0.0076) * Linha 7

Eliminando variaveis abaixo do pivo A(8, 8) = 3.3544926240500668:
Linha 9 = Linha 9 - (-0.3284) * Linha 8
```

```
--- FASE 2: SUBSTITUICAO REGRESSIVA ---
x(9) = 0.4286
x(8) = 0.5268
x(7) = 0.4286
x(6) = 0.1875
x(5) = 0.2500
x(4) = 0.1875
x(3) = 0.0714
x(2) = 0.0982
x(1) = 0.0714

---------------------------------------------
SOLUCAO FINAL ENCONTRADA:
Vetor x = (0.0714, 0.0982, 0.0714, 0.1875, 0.2500, 0.1875, 0.4286, 0.5268, 0.4286)
---------------------------------------------
```

Saída – Fatoração LU (Entrada 1)

```
====== RELATORIO DE SISTEMAS LINEARES ======
Metodo: Fatoracao LU
Tamanho do sistema: 9x9
==========================================

--- FASE 1: FATORACAO A = L * U ---

Matriz [L] (Inferior com os fatores):
    1.0000    0.0000    0.0000    0.0000    0.0000    0.0000    0.0000    0.0000    0.0000
   -0.2500    1.0000    0.0000    0.0000    0.0000    0.0000    0.0000    0.0000    0.0000
    0.0000   -0.2667    1.0000    0.0000    0.0000    0.0000    0.0000    0.0000    0.0000
   -0.2500   -0.0667   -0.0179    1.0000    0.0000    0.0000    0.0000    0.0000    0.0000
    0.0000   -0.2667   -0.0714   -0.2871    1.0000    0.0000    0.0000    0.0000    0.0000
    0.0000    0.0000   -0.2679   -0.0048   -0.3160    1.0000    0.0000    0.0000    0.0000
    0.0000    0.0000    0.0000   -0.2679   -0.0843   -0.0282    1.0000    0.0000    0.0000
    0.0000    0.0000    0.0000    0.0000   -0.2935   -0.0932   -0.2950    1.0000    0.0000
    0.0000    0.0000    0.0000    0.0000    0.0000   -0.2948   -0.0076   -0.3284    1.0000

Matriz [U] (Superior):
    4.0000   -1.0000    0.0000   -1.0000    0.0000    0.0000    0.0000    0.0000    0.0000
    0.0000    3.7500   -1.0000   -0.2500   -1.0000    0.0000    0.0000    0.0000    0.0000
    0.0000    0.0000    3.7333   -0.0667   -0.2667   -1.0000    0.0000    0.0000    0.0000
    0.0000    0.0000    0.0000    3.7321   -1.0714   -0.0179   -1.0000    0.0000    0.0000
    0.0000    0.0000    0.0000    0.0000    3.4067   -1.0766   -0.2871   -1.0000    0.0000
    0.0000    0.0000    0.0000    0.0000    0.0000    3.3919   -0.0955   -0.3160   -1.0000
    0.0000    0.0000    0.0000    0.0000    0.0000    0.0000    3.7052   -1.0932   -0.0282
    0.0000    0.0000    0.0000    0.0000    0.0000    0.0000    0.0000    3.3545   -1.1015
    0.0000    0.0000    0.0000    0.0000    0.0000    0.0000    0.0000    0.0000    3.3433

--- FASE 2: SUBSTITUICAO PROGRESSIVA (L * y = b) ---
y(1) = 0.0000
y(2) = 0.0000
y(3) = 0.0000
y(4) = 0.0000
y(5) = 0.0000
y(6) = 0.0000
y(7) = 1.0000
y(8) = 1.2950
y(9) = 1.4328

--- FASE 3: SUBSTITUICAO REGRESSIVA (U * x = y) ---
x(9) = 0.4286
x(8) = 0.5268
x(7) = 0.4286
x(6) = 0.1875
x(5) = 0.2500
x(4) = 0.1875
x(3) = 0.0714
x(2) = 0.0982
x(1) = 0.0714

---------------------------------------------
SOLUCAO FINAL ENCONTRADA:
Vetor x = (0.0714, 0.0982, 0.0714, 0.1875, 0.2500, 0.1875, 0.4286, 0.5268, 0.4286)
---------------------------------------------
```

Resposta 4.3: A probabilidade de o cachorro sair pelo lado sul estando na interseção i, sendo i índices de 1 a 9, é, respectivamente:

0.0714,

0.0982,

0.0714,

0.1875,

0.2500,

0.1875,

0.4286,

0.5268,

0.4286.

Problema Teste 3 - Página 153 Questão 4.6

4.6 - *Considere o circuito em escada, mostrado na Figura 4.7, composto de resistores concentrados e fontes de tensão ideais, uma independente e outra controlada.*

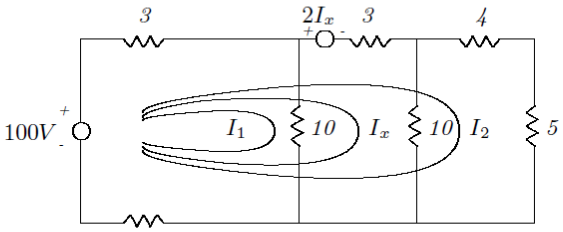


Figura 4.7

O sistema de equações que se obtém ao solucionar o circuito pelo método dos laços é o seguinte:

$$\begin{cases} 4\ (I_1 + I_2 + \ I_x) + 10\ I_1 = 100 \\ 4\ (I_1 + I_2 + I_x) + 3\ (I_2 + I_x) + 10\ I_x = 100 - 2\ I_x \\ 4\ (I_1 + I_2 + I_x) + 3\ (I_2 + I_x) + 9\ I_x = 100 - 2\ I_x \end{cases}$$

a) *Escreva o sistema acima na forma $Ax = b$.*

b) *Determine A^{-1}.*

c) *Usando o resultado obtido em* **b)** *determine o valor de I_x.*

Entrada 1 – para Eliminação de Gauss e Fatoração LU

```
3
14 4 4 100
4 7 19 100
4 7 18 100
```

Saída – Eliminação de Gauss (Entrada 1)

```
====== RELATORIO DE SISTEMAS LINEARES ======
Tamanho do sistema: 3x3
===========================================

--- FASE 1: ELIMINACAO PROGRESSIVA ---

Eliminando variaveis abaixo do pivo A(1, 1) = 14.0:
Linha 2 = Linha 2 - (0.2857) * Linha 1
Linha 3 = Linha 3 - (0.2857) * Linha 1

Eliminando variaveis abaixo do pivo A(2, 2) = 5.857142857142858:
Linha 3 = Linha 3 - (1.0000) * Linha 2

--- FASE 2: SUBSTITUICAO REGRESSIVA ---
x(3) = -0.0000
x(2) = 12.1951
x(1) = 3.6585

-------------------------------------------
SOLUCAO FINAL ENCONTRADA:
Vetor x = (3.6585, 12.1951, -0.0000)
-------------------------------------------
```

Saída – Fatoração LU (Entrada 1)

```
====== RELATORIO DE SISTEMAS LINEARES ======
Metodo: Fatoracao LU
Tamanho do sistema: 3x3
==========================================

--- FASE 1: FATORACAO A = L * U ---

Matriz [L] (Inferior com os fatores):
    1.0000    0.0000    0.0000
    0.2857    1.0000    0.0000
    0.2857    1.0000    1.0000

Matriz [U] (Superior):
   14.0000    4.0000    4.0000
    0.0000    5.8571   17.8571
    0.0000    0.0000   -1.0000

--- FASE 2: SUBSTITUICAO PROGRESSIVA (L * y = b) ---
y(1) = 100.0000
y(2) = 71.4286
y(3) = 0.0000

--- FASE 3: SUBSTITUICAO REGRESSIVA (U * x = y) ---
x(3) = -0.0000
x(2) = 12.1951
x(1) = 3.6585

------------------------------------------
SOLUCAO FINAL ENCONTRADA:
Vetor x = (3.6585, 12.1951, -0.0000)
------------------------------------------
```

Resposta 4.6: O valor de I_x é 0 ampères.

Método de Gauss-Seidel

Estratégia de Implementação:

O Método de Gauss-Seidel foi implementado como uma abordagem iterativa para a resolução de sistemas lineares, partindo de um palpite inicial nulo (um vetor de zeros). O grande diferencial na arquitetura deste código é a atualização imediata das variáveis: assim que um novo valor x(i) é calculado, ele já substitui o valor antigo no vetor principal e passa a ser utilizado imediatamente no cálculo da próxima variável x(i+1) dentro da mesma iteração. Isso acelera significativamente a convergência.

Para tornar o algoritmo mais robusto, foram implementadas duas lógicas essenciais:

(a) Prevenção de Divisão por Zero: Antes de iniciar o laço iterativo, o código varre a diagonal principal. Se encontrar um A(i, i) igual a zero, ele acusa "ERRO FATAL" e instrui o usuário a reorganizar as linhas do sistema, pois o método é matematicamente incapaz de prosseguir nessas condições.

(b) Fator de Relaxamento: O código introduz a variável "relaxamento" (lambda). A fórmula final aplica uma média ponderada entre o valor recém-calculado e o valor da iteração anterior. Se o sistema apresentar divergência (os valores começarem a explodir), o usuário pode diminuir esse fator para suavizar a transição dos valores e forçar a convergência.

Estrutura dos Arquivos de Entrada/Saída

O arquivo "entrada.txt" compartilha rigorosamente a mesma estrutura matricial utilizada nos métodos diretos (Gauss e LU), permitindo testar o mesmo sistema linear em algoritmos diferentes sem reescrever dados. O "saida.txt", no entanto, muda seu foco de relatório. Por ser iterativo, o arquivo registra os parâmetros iniciais (tolerância alvo e fator de relaxamento) e, em seguida, imprime um bloco para cada iteração, listando os novos valores calculados para cada variável e destacando o maior erro relativo encontrado naquela rodada. Ao atingir a convergência, a solução final é apresentada isolada e formatada entre parênteses, como em (x1, x2, x3).

Dificuldades enfrentadas

A maior dificuldade intrínseca aos métodos iterativos, e ao Gauss-Seidel em particular, é que a convergência não é garantida para qualquer matriz. O sistema precisa, idealmente, ser estritamente diagonal dominante, ou seja, o valor absoluto do elemento na diagonal principal de cada linha é maior ou igual à soma dos valores absolutos dos outros elementos dessa mesma linha. Caso contrário, o método pode divergir e atingir o limite máximo de iterações estipulado sem encontrar a raiz. Além disso, a dependência sequencial dos dados (um cálculo precisa do resultado exato do cálculo imediatamente anterior) dificulta a paralelização deste algoritmo em sistemas de alto desempenho.

Método de Jacobi

Estratégia de Implementação:

O Método de Jacobi compartilha a fundação iterativa do Gauss-Seidel (como a verificação de zeros na diagonal e o chute inicial nulo), mas diverge completamente na sua política de atualização de estado. A estratégia de implementação aqui utiliza uma cópia "congelada" do vetor. No início de cada iteração, o programa salva o estado atual em "x_antigo". Durante as equações, todos os novos valores de "x_novo" são calculados dependendo exclusivamente dos valores estáticos de "x_antigo". O vetor oficial só é atualizado de uma vez só, em bloco, no final do laço.

Essa independência entre as equações de uma mesma iteração é a característica que torna este método o preferido para execução em supercomputadores e placas de vídeo (GPUs), pois permite que todas as variáveis sejam calculadas simultaneamente (paralelismo). O controle de parada (convergência) também ganhou uma implementação ligeiramente diferente, adotando a norma global do vetor: o erro relativo é calculado dividindo a maior diferença absoluta encontrada entre as iterações pelo maior elemento absoluto do vetor atual.

Estrutura dos Arquivos de Entrada/Saída

A interface de entrada mantém a consistência arquitetural do projeto, utilizando a dimensão (n) seguida da matriz aumentada lida a partir do "entrada.txt". O relatório de saída no "saida.txt" espelha o visual do Gauss-Seidel para facilitar comparações diretas de desempenho. Ele evidencia o esforço iterativo, rodada a rodada, para que o usuário possa observar que, para o mesmo nível de tolerância, o método de Jacobi geralmente consome um número maior de iterações para convergir em comparação com o seu sucessor atualizado (Gauss-Seidel).

Dificuldades enfrentadas

As dificuldades enfrentadas com o método de Jacobi giram em torno da sua velocidade e dos mesmos critérios rígidos de convergência (necessidade

de matrizes diagonalmente dominantes). Por congelar os dados e só atualizá-los no final do ciclo, a sua taxa de convergência é visivelmente mais lenta que a do Gauss-Seidel. Ele requer mais iterações (e mais armazenamento na memória para manter as cópias dos vetores temporários) para atingir a mesma tolerância, o que o torna menos eficiente computacionalmente para processadores de núcleo único ou execução estritamente sequencial como no Python padrão.

Condicionamento de Matrizes

Estratégia de Implementação:

Diferente dos algoritmos anteriores, este código não tem o objetivo de resolver um sistema linear, mas sim de atuar como uma ferramenta de diagnóstico prévio. Ele avalia o quão sensível uma matriz é a pequenas perturbações (erros de arredondamento ou variações nos dados de entrada). A métrica central implementada é o Número de Condição, calculado pela fórmula: Cond(A) = ||A|| * ||A^-1||. Para isso, o código utiliza a Norma Infinita (também conhecida como Norma Linha), que busca a maior soma dos valores absolutos das linhas de uma matriz.

O grande destaque arquitetural desta implementação é a estratégia empregada para encontrar a matriz inversa (A^-1). Calcular inversas é um processo notoriamente pesado. Em vez de usar um método engessado, o algoritmo tira proveito máximo das propriedades da Fatoração LU. Sabendo que, após fatorar a matriz A nas matrizes L e U (Fase 1), o custo para calcular novos vetores independentes despenca, o código executa a pesada Fase 1 apenas uma única vez. A partir daí, para encontrar a inversa resolvendo A*x = I, o programa apenas roda as "leves" Fases 2 e 3 de substituição repetidas vezes, injetando as colunas da Matriz Identidade no lugar do vetor 'b'. Ao final, as normas são multiplicadas e o sistema emite um laudo automático: matriz bem condicionada (Cond < 100), moderada, ou mal condicionada (hipersensível).

Estrutura dos Arquivos de Entrada/Saída

A lógica de leitura no "entrada.txt" sofreu uma sutil adaptação: como não se trata da resolução de um sistema direto, o arquivo deve conter exclusivamente os coeficientes da matriz quadrada A. Não há vetor de termos independentes acoplado no final das linhas, e o algoritmo determina o tamanho da matriz (n) de forma autônoma através da contagem dinâmica de elementos nativa do Python. O arquivo "saida.txt" foi formatado como um laudo pericial contendo quatro passos descritivos. Ele guia o leitor pelo valor de cada norma calculada, imprime o status final da estabilidade do sistema numérico e, como

documentação de prova, imprime a matriz inversa recém-calculada com suas linhas encapsuladas por parênteses.

Dificuldades enfrentadas

A barreira fundamental na verificação do condicionamento é o custo operacional inerente ao processo matemático. Como a fórmula exige obrigatoriamente conhecer a matriz inversa completa (A^-1), fazer essa avaliação para sistemas muito grandes exige altíssimo poder de processamento computacional, quase superando o esforço de resolver o sistema linear em si. Além disso, o algoritmo herda as fragilidades da Fatoração LU sem pivoteamento: se a matriz principal for singular ou apresentar um elemento nulo na posição do pivô durante a decomposição L e U, o processo de inversão falha prematuramente. Nesse caso, o programa precisa interromper o cálculo de condicionamento por segurança, atestando a impossibilidade da análise.

Problema Teste 1 - Página 185 Questão 5.1

5.1 - *Uma maneira de se obter a solução da equação de Laplace:*

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0 \ ,$$

em uma região retangular consiste em se fazer uma discretização que transforma a equação em um problema aproximado consistindo em uma equação de diferenças cuja solução, em um caso particular, exige a solução do seguinte sistema linear:

$$\begin{pmatrix} 4 & -1 & 0 & -1 & 0 & 0 \\ -1 & 4 & -1 & 0 & -1 & 0 \\ 0 & -1 & 4 & 0 & 0 & -1 \\ -1 & 0 & 0 & 4 & -1 & 0 \\ 0 & -1 & 0 & -1 & 4 & -1 \\ 0 & 0 & -1 & 0 & -1 & 4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{pmatrix} = \begin{pmatrix} 100 \\ 0 \\ 0 \\ 100 \\ 0 \\ 0 \end{pmatrix} .$$

Se desejamos a solução com quatro algarismos significativos corretos, qual dos métodos iterativos que você conhece poderia ser aplicado com garantia de convergência? Resolva o sistema pelo método escolhido.

Entrada 1 – para Gauss-Seidel e Jacobi

```
6
4 -1 0 -1 0 0 100
-1 4 -1 0 -1 0 0
0 -1 4 0 0 -1 0
-1 0 0 4 -1 0 100
0 -1 0 -1 4 -1 0
0 0 -1 0 -1 4 0
```

Entrada 1 – para Condicionamento de Matriz

```
4 -1 0 -1 0 0
-1 4 -1 0 -1 0
0 -1 4 0 0 -1
-1 0 0 4 -1 0
0 -1 0 -1 4 -1
0 0 -1 0 -1 4
```

Saída – Gauss-Seidel (Entrada 1)

```
====== RELATORIO DE SISTEMAS LINEARES ======
Metodo: Gauss-Seidel
Tamanho do sistema: 6x6
==========================================

--- INICIO DO METODO DE GAUSS-SEIDEL ---
Tolerancia alvo: 0.01%
Fator de relaxamento: 1.0

--- Iteracao 1 ---
x(1) calculado = 25.000000
x(2) calculado = 6.250000
x(3) calculado = 1.562500
x(4) calculado = 31.250000
x(5) calculado = 9.375000
x(6) calculado = 2.734375
Maior erro relativo atual: 100.000000%

--- Iteracao 2 ---
x(1) calculado = 34.375000
x(2) calculado = 11.328125
x(3) calculado = 3.515625
x(4) calculado = 35.937500
x(5) calculado = 12.500000
x(6) calculado = 4.003906
Maior erro relativo atual: 55.555556%

--- Iteracao 3 ---
x(1) calculado = 36.816406
x(2) calculado = 13.208008
x(3) calculado = 4.302979
x(4) calculado = 37.329102
x(5) calculado = 13.635254
x(6) calculado = 4.484558
Maior erro relativo atual: 18.297872%

--- Iteracao 4 ---
x(1) calculado = 37.634277
x(2) calculado = 13.893127
x(3) calculado = 4.594421
x(4) calculado = 37.817383
x(5) calculado = 14.048767
x(6) calculado = 4.660797
Maior erro relativo atual: 6.343408%
```

```
--- Iteracao 5 ---
x(1) calculado = 37.927628
x(2) calculado = 14.142704
x(3) calculado = 4.700875
x(4) calculado = 37.994099
x(5) calculado = 14.199400
x(6) calculado = 4.725069
Maior erro relativo atual: 2.264555%

--- Iteracao 6 ---
x(1) calculado = 38.034201
x(2) calculado = 14.233619
x(3) calculado = 4.739672
x(4) calculado = 38.058400
x(5) calculado = 14.254272
x(6) calculado = 4.748486
Maior erro relativo atual: 0.818552%

--- Iteracao 7 ---
x(1) calculado = 38.073005
x(2) calculado = 14.266737
x(3) calculado = 4.753806
x(4) calculado = 38.081819
x(5) calculado = 14.274261
x(6) calculado = 4.757017
Maior erro relativo atual: 0.297316%

--- Iteracao 8 ---
x(1) calculado = 38.087139
x(2) calculado = 14.278801
x(3) calculado = 4.758954
x(4) calculado = 38.090350
x(5) calculado = 14.281542
x(6) calculado = 4.760124
Maior erro relativo atual: 0.108190%

--- Iteracao 9 ---
x(1) calculado = 38.092288
x(2) calculado = 14.283196
x(3) calculado = 4.760830
x(4) calculado = 38.093457
x(5) calculado = 14.284194
x(6) calculado = 4.761256
Maior erro relativo atual: 0.039396%
```

```
--- Iteracao 10 ---
x(1) calculado = 38.094163
x(2) calculado = 14.284797
x(3) calculado = 4.761513
x(4) calculado = 38.094589
x(5) calculado = 14.285161
x(6) calculado = 4.761668
Maior erro relativo atual: 0.014349%

--- Iteracao 11 ---
x(1) calculado = 38.094847
x(2) calculado = 14.285380
x(3) calculado = 4.761762
x(4) calculado = 38.095002
x(5) calculado = 14.285513
x(6) calculado = 4.761819
Maior erro relativo atual: 0.005227%

CONVERGENCIA ATINGIDA na iteracao 11!

---------------------------------------------
SOLUCAO FINAL ENCONTRADA:
Vetor x = (38.0948, 14.2854, 4.7618, 38.0950, 14.2855, 4.7618)
---------------------------------------------
```

Saída – Jacobi (Entrada 1)

```
====== RELATORIO DE SISTEMAS LINEARES ======
Metodo: Jacobi
Tamanho do sistema: 6x6
==========================================

--- INICIO DO METODO DE JACOBI ---
Tolerancia alvo: 0.01

--- Iteracao 1 ---
x(1) calculado = 25.000000
x(2) calculado = 0.000000
x(3) calculado = 0.000000
x(4) calculado = 25.000000
x(5) calculado = 0.000000
x(6) calculado = 0.000000
Erro relativo maximo atual: 1.000000

--- Iteracao 2 ---
x(1) calculado = 31.250000
x(2) calculado = 6.250000
x(3) calculado = 0.000000
x(4) calculado = 31.250000
x(5) calculado = 6.250000
x(6) calculado = 0.000000
Erro relativo maximo atual: 0.200000

--- Iteracao 3 ---
x(1) calculado = 34.375000
x(2) calculado = 9.375000
x(3) calculado = 1.562500
x(4) calculado = 34.375000
x(5) calculado = 9.375000
x(6) calculado = 1.562500
Erro relativo maximo atual: 0.090909

--- Iteracao 4 ---
x(1) calculado = 35.937500
x(2) calculado = 11.328125
x(3) calculado = 2.734375
x(4) calculado = 35.937500
x(5) calculado = 11.328125
x(6) calculado = 2.734375
Erro relativo maximo atual: 0.054348
```

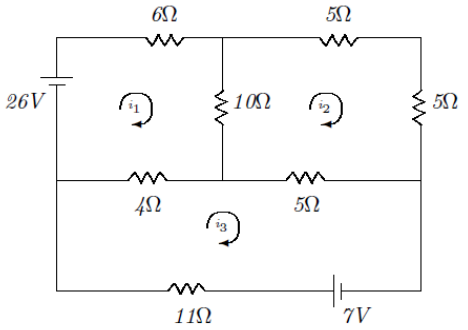
```
--- Iteracao 5 ---
x(1) calculado = 36.816406
x(2) calculado = 12.500000
x(3) calculado = 3.515625
x(4) calculado = 36.816406
x(5) calculado = 12.500000
x(6) calculado = 3.515625
Erro relativo maximo atual: 0.031830

--- Iteracao 6 ---
x(1) calculado = 37.329102
x(2) calculado = 13.208008
x(3) calculado = 4.003906
x(4) calculado = 37.329102
x(5) calculado = 13.208008
x(6) calculado = 4.003906
Erro relativo maximo atual: 0.018967

--- Iteracao 7 ---
x(1) calculado = 37.634277
x(2) calculado = 13.635254
x(3) calculado = 4.302979
x(4) calculado = 37.634277
x(5) calculado = 13.635254
x(6) calculado = 4.302979
Erro relativo maximo atual: 0.011353

--- Iteracao 8 ---
x(1) calculado = 37.817383
x(2) calculado = 13.893127
x(3) calculado = 4.484558
x(4) calculado = 37.817383
x(5) calculado = 13.893127
x(6) calculado = 4.484558
Erro relativo maximo atual: 0.006819

CONVERGENCIA ATINGIDA na iteracao 8!

--------------------------------------------
SOLUCAO FINAL ENCONTRADA:
Vetor x = (37.8174, 13.8931, 4.4846, 37.8174, 13.8931, 4.4846)
--------------------------------------------
```

Saída – Condicionamento de Matriz (Entrada 1)

```
====== RELATORIO DE CONDICIONAMENTO ======
Tamanho da matriz: 6x6
Norma utilizada: Norma Infinita (Norma Linha)
==============================================

--- PASSO 1: CALCULO DA NORMA DE [A] ---
Norma Infinita de A = 7.00000

--- PASSO 2: CALCULO DA MATRIZ INVERSA ---
Utilizando Fatoracao LU para resolver A*x = I
Matriz Inversa [A^-1] encontrada com sucesso.

--- PASSO 3: CALCULO DA NORMA DE [A^-1] ---
Norma Infinita de A^-1 = 0.71429

--- PASSO 4: NUMERO DE CONDICAO ---
Cond(A) = ||A|| * ||A^-1||
Cond(A) = 7.00000 * 0.71429
Cond(A) = 5.00000

--- AVALIACAO FINAL ---
Status: Matriz BEM CONDICIONADA. Solucoes serao estaveis.

--------------------------------------------------
VISUALIZACAO DA MATRIZ INVERSA GERADA:
( 0.29482,  0.09317,  0.02816,  0.08613,  0.04969,  0.01946)
( 0.09317,  0.32298,  0.09317,  0.04969,  0.10559,  0.04969)
( 0.02816,  0.09317,  0.29482,  0.01946,  0.04969,  0.08613)
( 0.08613,  0.04969,  0.01946,  0.29482,  0.09317,  0.02816)
( 0.04969,  0.10559,  0.04969,  0.09317,  0.32298,  0.09317)
( 0.01946,  0.04969,  0.08613,  0.02816,  0.09317,  0.29482)
--------------------------------------------------
```

Problema Teste 2 - Página 185 Questão 5.2

5.2 - *Considere o circuito da figura a seguir, com resistências e baterias tal como indicado; escolhemos arbitrariamente as orientações das correntes.*



Aplicando a lei de Kirchoff, que diz que a soma algébrica das diferenças de potencial em qualquer circuito fechado é zero, achamos para as correntes i_1, i_2, i_3:

$$\begin{cases} 6i_1 + 10(i_1 - i_2) + 4(i_1 - i_3) - 26 = 0 \\ 5i_2 + 5i_2 + 5(i_2 - i_3) + 10(i_2 - i_1) = 0 \\ 11i_3 + 4(i_3 - i_1) + 5(i_3 - i_2) - 7 = 0 \end{cases}$$

a) *É possível aplicar ao sistema acima o método de Gauss - Seidel com convergência assegurada? Justifique.*

b) *Se possível, obtenha a solução com erro relativo* $< 10^{-2}$.

Entrada 1 – para Gauss-Seidel e Jacobi

```
3
20 -10 -4 26
-10 25 -5 0
-4 -5 20 7
```

Entrada 1 – para Condicionamento de Matriz

```
20 -10 -4
-10 25 -5
-4 -5 20
```

Saída – Gauss-Seidel (Entrada 1)

```
====== RELATORIO DE SISTEMAS LINEARES ======
Metodo: Gauss-Seidel
Tamanho do sistema: 3x3
==========================================

--- INICIO DO METODO DE GAUSS-SEIDEL ---
Tolerancia alvo: 0.01%
Fator de relaxamento: 1.0

--- Iteracao 1 ---
x(1) calculado = 1.300000
x(2) calculado = 0.520000
x(3) calculado = 0.740000
Maior erro relativo atual: 100.000000%

--- Iteracao 2 ---
x(1) calculado = 1.708000
x(2) calculado = 0.831200
x(3) calculado = 0.899400
Maior erro relativo atual: 37.439846%

--- Iteracao 3 ---
x(1) calculado = 1.895480
x(2) calculado = 0.938072
x(3) calculado = 0.963614
Maior erro relativo atual: 11.392729%

--- Iteracao 4 ---
x(1) calculado = 1.961759
x(2) calculado = 0.977426
x(3) calculado = 0.986708
Maior erro relativo atual: 4.026321%

--- Iteracao 5 ---
x(1) calculado = 1.986055
x(2) calculado = 0.991764
x(3) calculado = 0.995152
Maior erro relativo atual: 1.445635%

--- Iteracao 6 ---
x(1) calculado = 1.994912
x(2) calculado = 0.996995
x(3) calculado = 0.998231
Maior erro relativo atual: 0.524741%
```

```
--- Iteracao 7 ---
x(1) calculado = 1.998144
x(2) calculado = 0.998904
x(3) calculado = 0.999355
Maior erro relativo atual: 0.191065%

--- Iteracao 8 ---
x(1) calculado = 1.999323
x(2) calculado = 0.999600
x(3) calculado = 0.999765
Maior erro relativo atual: 0.069656%

--- Iteracao 9 ---
x(1) calculado = 1.999753
x(2) calculado = 0.999854
x(3) calculado = 0.999914
Maior erro relativo atual: 0.025406%

--- Iteracao 10 ---
x(1) calculado = 1.999910
x(2) calculado = 0.999947
x(3) calculado = 0.999969
Maior erro relativo atual: 0.009268%

CONVERGENCIA ATINGIDA na iteracao 10!

---------------------------------------------
SOLUCAO FINAL ENCONTRADA:
Vetor x = (1.9999, 0.9999, 1.0000)
---------------------------------------------
```

Saída – Jacobi (Entrada 1)

```
====== RELATORIO DE SISTEMAS LINEARES ======
Metodo: Jacobi
Tamanho do sistema: 3x3
==========================================

--- INICIO DO METODO DE JACOBI ---
Tolerancia alvo: 0.01

--- Iteracao 1 ---
x(1) calculado = 1.300000
x(2) calculado = 0.000000
x(3) calculado = 0.350000
Erro relativo maximo atual: 1.000000

--- Iteracao 2 ---
x(1) calculado = 1.370000
x(2) calculado = 0.590000
x(3) calculado = 0.610000
Erro relativo maximo atual: 0.430657

--- Iteracao 3 ---
x(1) calculado = 1.717000
x(2) calculado = 0.670000
x(3) calculado = 0.771500
Erro relativo maximo atual: 0.202097

--- Iteracao 4 ---
x(1) calculado = 1.789300
x(2) calculado = 0.841100
x(3) calculado = 0.860900
Erro relativo maximo atual: 0.095624

--- Iteracao 5 ---
x(1) calculado = 1.892730
x(2) calculado = 0.887900
x(3) calculado = 0.918135
Erro relativo maximo atual: 0.054646

--- Iteracao 6 ---
x(1) calculado = 1.927577
x(2) calculado = 0.940719
x(3) calculado = 0.950521
Erro relativo maximo atual: 0.027402
```

```
--- Iteracao 7 ---
x(1) calculado = 1.960464
x(2) calculado = 0.961135
x(3) calculado = 0.970695
Erro relativo maximo atual: 0.016775

--- Iteracao 8 ---
x(1) calculado = 1.974707
x(2) calculado = 0.978325
x(3) calculado = 0.982376
Erro relativo maximo atual: 0.008705

CONVERGENCIA ATINGIDA na iteracao 8!

---------------------------------------------
SOLUCAO FINAL ENCONTRADA:
Vetor x = (1.9747, 0.9783, 0.9824)
---------------------------------------------
```

Saída – Condicionamento de Matriz (Entrada 1)

```
====== RELATORIO DE CONDICIONAMENTO ======
Tamanho da matriz: 3x3
Norma utilizada: Norma Infinita (Norma Linha)
=============================================

--- PASSO 1: CALCULO DA NORMA DE [A] ---
Norma Infinita de A = 40.00000

--- PASSO 2: CALCULO DA MATRIZ INVERSA ---
Utilizando Fatoracao LU para resolver A*x = I
Matriz Inversa [A^-1] encontrada com sucesso.

--- PASSO 3: CALCULO DA NORMA DE [A^-1] ---
Norma Infinita de A^-1 = 0.12612

--- PASSO 4: NUMERO DE CONDICAO ---
Cond(A) = ||A|| * ||A^-1||
Cond(A) = 40.00000 * 0.12612
Cond(A) = 5.04478

--- AVALIACAO FINAL ---
Status: Matriz BEM CONDICIONADA. Solucoes serao estaveis.

-------------------------------------------------
VISUALIZACAO DA MATRIZ INVERSA GERADA:
( 0.07090,  0.03284,  0.02239)
( 0.03284,  0.05731,  0.02090)
( 0.02239,  0.02090,  0.05970)
-------------------------------------------------
```

Resposta 5.2: A solução para o problema são os valores de i, que são, aproximadamente:

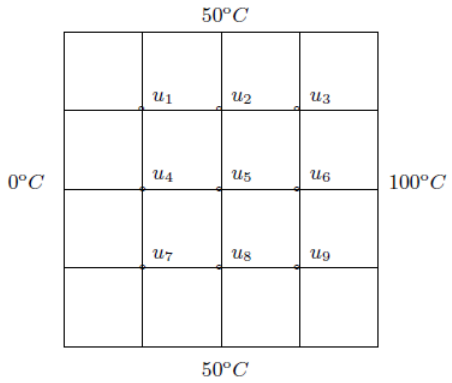
$i_1 = 2$

$i_2 = 1$

$i_3 = 1$

Problema Teste 3 - Página 188 Questão 5.5

5.5 - *Suponha que uma membrana com dimensões 80 cm × 80 cm tenha cada um dos seus lados mantidos a uma temperatura constante. Usando a teoria de equações diferenciais parciais pode-se formular uma equação que determina o valor da temperatura no interior dessa membrana. Essa equação diferencial pode ser simplificada colocando-se uma malha com 9 pontos sobre essa membrana e calculando-se a temperatura nos pontos da malha, como mostra a figura:*



onde $u_1, u_2, ..., u_9$ são os valores da temperatura em cada ponto da malha.

O sistema de equações resultante é dado por:

$$\begin{pmatrix} -4 & 1 & & 1 & & & & & \\ 1 & -4 & 1 & & 1 & & & & \\ & 1 & -4 & & & 1 & & & \\ 1 & & & -4 & 1 & & 1 & & \\ & 1 & & 1 & -4 & 1 & & 1 & \\ & & 1 & & 1 & -4 & & & 1 \\ & & & & 1 & & -4 & 1 & \\ & & & & 1 & & 1 & -4 & 1 \\ & & & & & 1 & & 1 & -4 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \\ u_8 \\ u_9 \end{pmatrix} = \begin{pmatrix} -50 \\ -50 \\ -150 \\ 0 \\ 0 \\ -100 \\ -50 \\ -50 \\ -150 \end{pmatrix}.$$

Encontre a solução do sistema por método numérico à sua escolha, com garantia de convergência.

Entrada 1 – para Gauss-Seidel e Jacobi

```
9
-4 1 0 1 0 0 0 0 0 -50
1 -4 1 0 1 0 0 0 0 -50
0 1 -4 0 0 1 0 0 0 -150
1 0 0 -4 1 0 1 0 0 0
0 1 0 1 -4 1 0 1 0 0
0 0 1 0 1 -4 0 0 1 -100
0 0 0 1 0 0 -4 1 0 -50
0 0 0 0 1 0 1 -4 1 -50
0 0 0 0 0 1 0 1 -4 -150
```

Entrada 1 – para Condicionamento de Matriz

```
-4 1 0 1 0 0 0 0 0
1 -4 1 0 1 0 0 0 0
0 1 -4 0 0 1 0 0 0
1 0 0 -4 1 0 1 0 0
0 1 0 1 -4 1 0 1 0
0 0 1 0 1 -4 0 0 1
0 0 0 1 0 0 -4 1 0
0 0 0 0 1 0 1 -4 1
0 0 0 0 0 1 0 1 -4
```

Saída – Gauss-Seidel (Entrada 1)

```
====== RELATORIO DE SISTEMAS LINEARES ======
Metodo: Gauss-Seidel
Tamanho do sistema: 9x9
==========================================

--- INICIO DO METODO DE GAUSS-SEIDEL ---
Tolerancia alvo: 0.01%
Fator de relaxamento: 1.0

--- Iteracao 1 ---
x(1) calculado = 12.500000
x(2) calculado = 15.625000
x(3) calculado = 41.406250
x(4) calculado = 3.125000
x(5) calculado = 4.687500
x(6) calculado = 36.523438
x(7) calculado = 13.281250
x(8) calculado = 16.992188
x(9) calculado = 50.878906
Maior erro relativo atual: 100.000000%

--- Iteracao 2 ---
x(1) calculado = 17.187500
x(2) calculado = 28.320312
x(3) calculado = 53.710938
x(4) calculado = 8.789062
x(5) calculado = 22.656250
x(6) calculado = 56.811523
x(7) calculado = 18.945312
x(8) calculado = 35.620117
x(9) calculado = 60.607910
Maior erro relativo atual: 79.310345%

--- Iteracao 3 ---
x(1) calculado = 21.777344
x(2) calculado = 37.036133
x(3) calculado = 60.961914
x(4) calculado = 15.844727
x(5) calculado = 36.328125
x(6) calculado = 64.474487
x(7) calculado = 25.366211
x(8) calculado = 43.075562
x(9) calculado = 64.387512
Maior erro relativo atual: 44.530046%

--- Iteracao 4 ---
x(1) calculado = 25.720215
x(2) calculado = 43.252563
x(3) calculado = 64.431763
x(4) calculado = 21.853638
x(5) calculado = 43.164062
x(6) calculado = 67.995834
x(7) calculado = 28.732300
x(8) calculado = 46.570969
x(9) calculado = 66.141701
Maior erro relativo atual: 27.496160%
```

```
--- Iteracao 5 ---
x(1) calculado = 28.776550
x(2) calculado = 46.593094
x(3) calculado = 66.147232
x(4) calculado = 25.168228
x(5) calculado = 46.582031
x(6) calculado = 69.717741
x(7) calculado = 30.434799
x(8) calculado = 48.289633
x(9) calculado = 67.001843
Maior erro relativo atual: 13.169741%

--- Iteracao 6 ---
x(1) calculado = 30.440331
x(2) calculado = 48.292398
x(3) calculado = 67.002535
x(4) calculado = 26.864290
x(5) calculado = 48.291016
x(6) calculado = 70.573848
x(7) calculado = 31.288481
x(8) calculado = 49.145335
x(9) calculado = 67.429796
Maior erro relativo atual: 6.313445%

--- Iteracao 7 ---
x(1) calculado = 31.289172
x(2) calculado = 49.145681
x(3) calculado = 67.429882
x(4) calculado = 27.717167
x(5) calculado = 49.145508
x(6) calculado = 71.001296
x(7) calculado = 31.715626
x(8) calculado = 49.572732
x(9) calculado = 67.643507
Maior erro relativo atual: 3.077071%

--- Iteracao 8 ---
x(1) calculado = 31.715712
x(2) calculado = 49.572776
x(3) calculado = 67.643518
x(4) calculado = 28.144211
x(5) calculado = 49.572754
x(6) calculado = 71.214945
x(7) calculado = 31.929236
x(8) calculado = 49.786374
x(9) calculado = 67.750330
Maior erro relativo atual: 1.517343%
```

```
--- Iteracao 9 ---
x(1) calculado = 31.929247
x(2) calculado = 49.786380
x(3) calculado = 67.750331
x(4) calculado = 28.357809
x(5) calculado = 49.786377
x(6) calculado = 71.321759
x(7) calculado = 32.036046
x(8) calculado = 49.893188
x(9) calculado = 67.803737
Maior erro relativo atual: 0.753224%

--- Iteracao 10 ---
x(1) calculado = 32.036047
x(2) calculado = 49.893189
x(3) calculado = 67.803737
x(4) calculado = 28.464617
x(5) calculado = 49.893188
x(6) calculado = 71.375166
x(7) calculado = 32.089451
x(8) calculado = 49.946594
x(9) calculado = 67.830440
Maior erro relativo atual: 0.375232%

--- Iteracao 11 ---
x(1) calculado = 32.089452
x(2) calculado = 49.946594
x(3) calculado = 67.830440
x(4) calculado = 28.518023
x(5) calculado = 49.946594
x(6) calculado = 71.401869
x(7) calculado = 32.116154
x(8) calculado = 49.973297
x(9) calculado = 67.843791
Maior erro relativo atual: 0.187269%

--- Iteracao 12 ---
x(1) calculado = 32.116154
x(2) calculado = 49.973297
x(3) calculado = 67.843791
x(4) calculado = 28.544726
x(5) calculado = 49.973297
x(6) calculado = 71.415220
x(7) calculado = 32.129506
x(8) calculado = 49.986649
x(9) calculado = 67.850467
Maior erro relativo atual: 0.093547%
```

```
--- Iteracao 13 ---
x(1) calculado = 32.129506
x(2) calculado = 49.986649
x(3) calculado = 67.850467
x(4) calculado = 28.558077
x(5) calculado = 49.986649
x(6) calculado = 71.421896
x(7) calculado = 32.136181
x(8) calculado = 49.993324
x(9) calculado = 67.853805
Maior erro relativo atual: 0.046752%

--- Iteracao 14 ---
x(1) calculado = 32.136181
x(2) calculado = 49.993324
x(3) calculado = 67.853805
x(4) calculado = 28.564753
x(5) calculado = 49.993324
x(6) calculado = 71.425234
x(7) calculado = 32.139519
x(8) calculado = 49.996662
x(9) calculado = 67.855474
Maior erro relativo atual: 0.023370%

--- Iteracao 15 ---
x(1) calculado = 32.139519
x(2) calculado = 49.996662
x(3) calculado = 67.855474
x(4) calculado = 28.568091
x(5) calculado = 49.996662
x(6) calculado = 71.426902
x(7) calculado = 32.141188
x(8) calculado = 49.998331
x(9) calculado = 67.856308
Maior erro relativo atual: 0.011684%

--- Iteracao 16 ---
x(1) calculado = 32.141188
x(2) calculado = 49.998331
x(3) calculado = 67.856308
x(4) calculado = 28.569760
x(5) calculado = 49.998331
x(6) calculado = 71.427737
x(7) calculado = 32.142023
x(8) calculado = 49.999166
x(9) calculado = 67.856726
Maior erro relativo atual: 0.005842%

CONVERGENCIA ATINGIDA na iteracao 16!

---------------------------------------------
SOLUCAO FINAL ENCONTRADA:
Vetor x = (32.1412, 49.9983, 67.8563, 28.5698, 49.9983, 71.4277, 32.1420, 49.9992, 67.8567)
---------------------------------------------
```

Saída – Jacobi (Entrada 1)

```
====== RELATORIO DE SISTEMAS LINEARES ======
Metodo: Jacobi
Tamanho do sistema: 9x9
==========================================

--- INICIO DO METODO DE JACOBI ---
Tolerancia alvo: 0.01

--- Iteracao 1 ---
x(1) calculado = 12.500000
x(2) calculado = 12.500000
x(3) calculado = 37.500000
x(4) calculado = -0.000000
x(5) calculado = -0.000000
x(6) calculado = 25.000000
x(7) calculado = 12.500000
x(8) calculado = 12.500000
x(9) calculado = 37.500000
Erro relativo maximo atual: 1.000000

--- Iteracao 2 ---
x(1) calculado = 15.625000
x(2) calculado = 25.000000
x(3) calculado = 46.875000
x(4) calculado = 6.250000
x(5) calculado = 12.500000
x(6) calculado = 43.750000
x(7) calculado = 15.625000
x(8) calculado = 25.000000
x(9) calculado = 46.875000
Erro relativo maximo atual: 0.400000

--- Iteracao 3 ---
x(1) calculado = 20.312500
x(2) calculado = 31.250000
x(3) calculado = 54.687500
x(4) calculado = 10.937500
x(5) calculado = 25.000000
x(6) calculado = 51.562500
x(7) calculado = 20.312500
x(8) calculado = 31.250000
x(9) calculado = 54.687500
Erro relativo maximo atual: 0.228571

--- Iteracao 4 ---
x(1) calculado = 23.046875
x(2) calculado = 37.500000
x(3) calculado = 58.203125
x(4) calculado = 16.406250
x(5) calculado = 31.250000
x(6) calculado = 58.593750
x(7) calculado = 23.046875
x(8) calculado = 37.500000
x(9) calculado = 58.203125
Erro relativo maximo atual: 0.120000
```

```
--- Iteracao 5 ---
x(1) calculado = 25.976562
x(2) calculado = 40.625000
x(3) calculado = 61.523438
x(4) calculado = 19.335938
x(5) calculado = 37.500000
x(6) calculado = 61.914062
x(7) calculado = 25.976562
x(8) calculado = 40.625000
x(9) calculado = 61.523438
Erro relativo maximo atual: 0.100946

--- Iteracao 6 ---
x(1) calculado = 27.490234
x(2) calculado = 43.750000
x(3) calculado = 63.134766
x(4) calculado = 22.363281
x(5) calculado = 40.625000
x(6) calculado = 65.136719
x(7) calculado = 27.490234
x(8) calculado = 43.750000
x(9) calculado = 63.134766
Erro relativo maximo atual: 0.049475

--- Iteracao 7 ---
x(1) calculado = 29.028320
x(2) calculado = 45.312500
x(3) calculado = 64.721680
x(4) calculado = 23.901367
x(5) calculado = 43.750000
x(6) calculado = 66.723633
x(7) calculado = 29.028320
x(8) calculado = 45.312500
x(9) calculado = 64.721680
Erro relativo maximo atual: 0.046835

--- Iteracao 8 ---
x(1) calculado = 29.803467
x(2) calculado = 46.875000
x(3) calculado = 65.509033
x(4) calculado = 25.451660
x(5) calculado = 45.312500
x(6) calculado = 68.298340
x(7) calculado = 29.803467
x(8) calculado = 46.875000
x(9) calculado = 65.509033
Erro relativo maximo atual: 0.023056
```

```
--- Iteracao 9 ---
x(1) calculado = 30.581665
x(2) calculado = 47.656250
x(3) calculado = 66.293335
x(4) calculado = 26.229858
x(5) calculado = 46.875000
x(6) calculado = 69.082642
x(7) calculado = 30.581665
x(8) calculado = 47.656250
x(9) calculado = 66.293335
Erro relativo maximo atual: 0.022618

--- Iteracao 10 ---
x(1) calculado = 30.971527
x(2) calculado = 48.437500
x(3) calculado = 66.684723
x(4) calculado = 27.009583
x(5) calculado = 47.656250
x(6) calculado = 69.865417
x(7) calculado = 30.971527
x(8) calculado = 48.437500
x(9) calculado = 66.684723
Erro relativo maximo atual: 0.011204

--- Iteracao 11 ---
x(1) calculado = 31.361771
x(2) calculado = 48.828125
x(3) calculado = 67.075729
x(4) calculado = 27.399826
x(5) calculado = 48.437500
x(6) calculado = 70.256424
x(7) calculado = 31.361771
x(8) calculado = 48.828125
x(9) calculado = 67.075729
Erro relativo maximo atual: 0.011120

--- Iteracao 12 ---
x(1) calculado = 31.556988
x(2) calculado = 49.218750
x(3) calculado = 67.271137
x(4) calculado = 27.790260
x(5) calculado = 48.828125
x(6) calculado = 70.647240
x(7) calculado = 31.556988
x(8) calculado = 49.218750
x(9) calculado = 67.271137
Erro relativo maximo atual: 0.005532

CONVERGENCIA ATINGIDA na iteracao 12!

---------------------------------------------
SOLUCAO FINAL ENCONTRADA:
Vetor x = (31.5570, 49.2188, 67.2711, 27.7903, 48.8281, 70.6472, 31.5570, 49.2188, 67.2711)
---------------------------------------------
```

Saída – Condicionamento de Matriz (Entrada 1)

```
====== RELATORIO DE CONDICIONAMENTO ======
Tamanho da matriz: 9x9
Norma utilizada: Norma Infinita (Norma Linha)
==============================================

--- PASSO 1: CALCULO DA NORMA DE [A] ---
Norma Infinita de A = 8.00000

--- PASSO 2: CALCULO DA MATRIZ INVERSA ---
Utilizando Fatoracao LU para resolver A*x = I
Matriz Inversa [A^-1] encontrada com sucesso.

--- PASSO 3: CALCULO DA NORMA DE [A^-1] ---
Norma Infinita de A^-1 = 1.12500

--- PASSO 4: NUMERO DE CONDICAO ---
Cond(A) = ||A|| * ||A^-1||
Cond(A) = 8.00000 * 1.12500
Cond(A) = 9.00000

--- AVALIACAO FINAL ---
Status: Matriz BEM CONDICIONADA. Solucoes serao estaveis.

--------------------------------------------------
VISUALIZACAO DA MATRIZ INVERSA GERADA:
(-0.29911, -0.09821, -0.03125, -0.09821, -0.06250, -0.02679, -0.03125, -0.02679, -0.01339)
(-0.09821, -0.33036, -0.09821, -0.06250, -0.12500, -0.06250, -0.02679, -0.04464, -0.02679)
(-0.03125, -0.09821, -0.29911, -0.02679, -0.06250, -0.09821, -0.01339, -0.02679, -0.03125)
(-0.09821, -0.06250, -0.02679, -0.33036, -0.12500, -0.04464, -0.09821, -0.06250, -0.02679)
(-0.06250, -0.12500, -0.06250, -0.12500, -0.37500, -0.12500, -0.06250, -0.12500, -0.06250)
(-0.02679, -0.06250, -0.09821, -0.04464, -0.12500, -0.33036, -0.02679, -0.06250, -0.09821)
(-0.03125, -0.02679, -0.01339, -0.09821, -0.06250, -0.02679, -0.29911, -0.09821, -0.03125)
(-0.02679, -0.04464, -0.02679, -0.06250, -0.12500, -0.06250, -0.09821, -0.33036, -0.09821)
(-0.01339, -0.02679, -0.03125, -0.02679, -0.06250, -0.09821, -0.03125, -0.09821, -0.29911)
--------------------------------------------------
```

Resposta 5.5: A solução do sistema é o conjunto de valores das temperaturas em cada ponto da malha, que são, aproximadamente:

u_1 = 32,0 ºC

u_2 = 49,5 ºC

u_3 = 67,5 ºC

u_4 = 28,2 ºC

u_5 = 49,4 ºC

u_6 = 71,0 ºC

u_7 = 31,8 ºC

u_8 = 49,5 ºC

u_9 = 67,5 ºC

Considerações Finais

O presente relatório documentou a implementação, o teste e a análise de métodos numéricos clássicos focados em dois dos problemas mais fundamentais da computação científica: a busca de raízes de funções reais e a resolução de sistemas de equações lineares. Através da linguagem Python, foi possível estruturar um laboratório de testes robusto e modular, separando a lógica matemática em arquivos de entrada e saída, e utilizando recursos da linguagem para generalizar a leitura de equações complexas em tempo de execução.

Na primeira etapa do projeto, dedicada às raízes de funções, a transição entre os métodos implementados evidenciou o clássico dilema da análise numérica: o balanço entre segurança e eficiência. Os métodos de confinamento (Bissecção e Falsa Posição) provaram ser ferramentas extremamente seguras, garantindo a convergência desde que o intervalo inicial respeite a troca de sinais. Em contrapartida, os métodos abertos (Newton-Raphson e Secante) demonstraram uma superioridade drástica na velocidade de convergência, exigindo muito menos iterações para atingir a tolerância exigida, embora trouxessem o ônus da vulnerabilidade à divergência em pontos de inclinação nula ou palpites iniciais ruins. A aplicação desses algoritmos na determinação do tempo de subida de circuitos amplificadores, cálculo de taxas de juros em financiamentos e na inclinação de trajetórias balísticas demonstrou a ponte vital entre as raízes matemáticas e a modelagem do mundo físico.

Na segunda etapa, voltada aos sistemas lineares, a implementação permitiu um contraste claro entre abordagens diretas e iterativas. A Eliminação de Gauss e a Fatoração LU mostraram-se métodos exatos e poderosos. A Fatoração LU, em especial, destacou-se pela sua inteligência computacional ao separar a matriz de coeficientes, permitindo a rápida resolução de sistemas com diferentes vetores independentes sem a necessidade de reprocessar a parte mais pesada do cálculo. Por outro lado, a implementação do Gauss-Seidel e do Jacobi revelou o cenário onde os métodos diretos perdem espaço: sistemas massivos e esparsos. A aplicação do Gauss-Seidel na resolução das probabilidades de caminhos em um labirinto e no cálculo da

distribuição térmica em uma membrana provou que matrizes diagonalmente dominantes são resolvidas de forma incrivelmente rápida e com menor gasto de memória pelas abordagens iterativas.

Além da resolução em si, a introdução da rotina de Análise de Condicionamento atuou como um marco de maturidade no projeto. A capacidade de reaproveitar a Fatoração LU para encontrar a matriz inversa e, a partir das normas infinitas, calcular o Número de Condição, reforça a ideia de que encontrar uma solução matemática não é o suficiente; é imperativo atestar a estabilidade do sistema e prever como ele reagirá a erros de arredondamento e perturbações externas.

Em suma, o desenvolvimento e a análise destes oito algoritmos consolidaram não apenas a teoria abordada na disciplina de Análise Numérica, mas também o raciocínio lógico necessário para tratar exceções em código (como travas contra divisão por zero) e a capacidade de traduzir enunciados práticos em modelos matemáticos solucionáveis por máquinas.